

SMT-Tutorial-for-AI-Class-3

May 7, 2026

1 Statistical Machine Translation Tutorial (with Normalization)

Ye Kyaw Thu, Lab Leader, Language Understanding Lab., Myanmar
For AI Fundamental Class students
Date: 3 May 2026

1.1 Data

```
[2]: %cd /home/ye/exp/SMT-NMT_tutorial
```

```
/home/ye/exp/SMT-NMT_tutorial
```

```
[6]: %cd ..
```

```
/home/ye/exp/SMT-NMT_tutorial
```

```
[19]: %cd clean-data/big
```

```
/home/ye/exp/SMT-NMT_tutorial/clean-data/big
```

```
[21]: !ls --color=auto
```

```
dev.my dev.th escape test.my test-sgm test.th  
train.my train.th
```

escape folder က ရှေ့က SMT-Tutorial-for-AI-Class-2 notebook တုန်းက error ပေးတဲ့ special character “<” တွေကို ရှင်းတုန်းက ဆောက်ထားခဲ့တဲ့ ဖိုင်ဒါပါ။ Escape လုပ်ထားပြီးသား ရှင်းထားတဲ့ ဖိုင်တွေက လက်ရှိ ./big/ ဖိုင်ဒါအောက်မှာ ကော်ပြီကူးထားပြီးသားပါ။ အခုတခေါက် experiment မှာလည်း အဲဒီ escape လုပ်ထားတဲ့ ဒေတာတွေကိုပဲ ယူသုံးမှ error မပေးမှာ။

ရှေ့က result ကို baseline အဖြစ်ထားကြရအောင်။

ဒီတစ်ခေါက် tutorial မှာတော့ မြန်မာ ဝဏ္ဏတွေရဲ့ အဓိက ပြဿနာ တစ်ခုဖြစ်တဲ့ normalization အပိုင်းကို သင်ပေးပါမယ်။

```
[24]: !mkdir normalize
```

```
[25]: !cp *.{my,th} ./normalize
```

```
[26]: %cd ./normalize
```

```
/home/ye/exp/SMT-NMT_tutorial/clean-data/big/normalize
```

```
[27]: !wc *.my
```

```
10000 117871 1127057 dev.my
 8000  93908  896438 test.my
62625 734667 7029791 train.my
80625 946446 9053286 total
```

```
[28]: !wc *.th
```

```
10000  71013  826135 dev.th
 8000   56113  652038 test.th
62625 439436 5111625 train.th
80625 566562 6589798 total
```

1.2 Why Normalize

အောက်ပါလိုမျိုး typing order တွေရှိနေလို့ပါ။

```
[30]: !head train.my
```

```
အောက် ပါ ရော ဂါ လက္ခဏာ တွေ များ ရှိ ရင်
စိတ် ကို အေး အေး ဆေး ဆေး ထား ပါ
နာ တာ ရှည် ရော ဂါ ရယ် လို့ မ ရှိ ပါ ဘူး ဗျ
လက္ခဏာ တွေ နဲ့
ပြီး တဲ့ အ ခါ နား ကြပ် ကို တံ တော ငဲ့ ဆစ် နား မှာ ထား မှာ မို့ ပါ ရှင်
ပု စွန် တော့ ကျွန် တော် စား တယ်
သို့ ပေ မဲ့ လည်း
ကျွန် တော် ခေါင်း ပေါ် ရေ ခဲ တင် ထား ရ မ လား
ကွန် ပြူ တာ ရောင် ခြည် နဲ့ ထပ် စစ် ဆေး ပါ မယ် နော်
ဒီ ကြား မှာ ခ ဏ လေး ထိုင် စော ငဲ့ ပေး လို့ ရ မ လား
```

Typing order မှားရိတ်ထားလို့ RE based syllable segmenter (sylbreak) နဲ့ ဖြတ်တဲ့အခါမှာ အောက်ပါလိုမျိုး error တွေ တက်ပါတယ်။

```
တော ငဲ့
စော ငဲ့
```

အဲဒီလိုမျိုး ပြဿနာတွေကို ဖြေရှင်းဖို့အတွက် syl_normalizer.py code လက်ရှိ ဆရာတို့ Lab မှာ သုံးနေတဲ့ ဗားရှင်းကိုပဲ သုံးကြရအောင်။

1.3 Syllable Normalizer

```
[32]: %ls ../../../../syl_normalizer/
```

```
myMono_syl_trigram.arpa  syl_normalizer_0.2/
syl_normalizer_0.5/
note.txt                  syl_normalizer_0.3/
```

```
syl_normalizer_0.6/  
syl_normalizer_0.1/      syl_normalizer_0.4/  
syl_normalizer2024/
```

syl_normalizer_0.6 မိုင်းဒါကို SMT-NMT_tutorial အောက်ကို ကော်ပီကူးယူမယ်။

```
[36]: %cd /home/ye/exp/SMT-NMT_tutorial
```

```
/home/ye/exp/SMT-NMT_tutorial
```

```
[37]: %cd syl_normalizer_0.6
```

```
/home/ye/exp/SMT-NMT_tutorial/syl_normalizer_0.6
```

```
"""
```

```
Burmese Syllable Normalizer
```

```
=====
```

Normalizes Myanmar text using a multi-stage pipeline:

Stage 0 - Unicode NFC normalization

Stage 1 - Regex-based rule corrections (medial ordering, vowel fixes, etc.)

Stage 2 - Fuzzy correction:

(a) TargetedFuzzyCorrector - checks candidates against the syllable frequency dictionary (--fuzzy-distance 1, no --ngram-lm)

(b) NgramFuzzyCorrector - scores candidates with an ARPA n-gram language model and accepts only if the candidate is strictly more probable than the original token (--ngram-lm)

Stage 3 - Consonant+Asat merge with previous syllable

Stage 4 - Compound syllable splitting (DP, 3 parts)

Written by Ye Kyaw Thu, LU Lab., Myanmar.

Version: 0.6

Changes from 0.5:

- * Stage 2 fuzzy correction now optionally driven by an ARPA n-gram LM (--ngram-lm). Candidates are scored with Katz backoff using up to 2 tokens of left context; a correction is accepted only when the candidate scores strictly better than the original token, preventing spurious corrections on foreign-name transliterations.*
- * Log now records every individual regex rule application with its rule ID and a short description for detailed error analysis.*
- * New --min-lm-improve flag (log10 units, default 0.5): candidate must be at least this much more probable than the original before the LM correction fires.*
- * New NgramLanguageModel class: loads ARPA trigram file, provides score_word(word, context) via Katz backoff.*
- * Backward-compatible: all v0.5 flags continue to work unchanged.*

Usage:

```
# Stage 1 + merge/split only (safest):
python3 syl_normalizer.py \
    --dictionary syl_dict.txt --frequency 2 \
    --input in.syl --output out.syl \
    --log corrections.log --error-output errors.txt \
    --fuzzy-distance 0

# Add n-gram LM-guided fuzzy correction:
python3 syl_normalizer.py \
    --dictionary syl_dict.txt --frequency 2 \
    --ngram-lm myMono_syl_trigram.arpa \
    --min-lm-improve 0.5 \
    --input in.syl --output out.syl \
    --log corrections.log --error-output errors.txt

# Fallback dict-based fuzzy (v0.5 behaviour):
python3 syl_normalizer.py \
    --dictionary syl_dict.txt --frequency 2 \
    --fuzzy-distance 1 \
    --input in.syl --output out.syl \
    --log corrections.log --error-output errors.txt
"""

import re
import sys
import math
import argparse
import unicodedata
from typing import Dict, List, Optional, Tuple

# =====
# Myanmar Unicode Constants
# =====

VIRAMA = '\u1039' # ၵ Stacking virama (sa-lon)
ASAT = '\u103A' # ၶ Asat (killer vowel)

MYANMAR_DIGITS = set(chr(cp) for cp in range(0x1040, 0x104A))
COMBINING_MARKS_RANGE = (0x102B, 0x103E)

# =====
# Confused Character Pairs
# =====
```

```

# Codepoint quick-reference (Unicode Standard, Myanmar block U1000):
#   U+1000=က   U+1001=ခ   U+1002=ဂ   U+1003=ဃ   U+1004=င   U+1005=စ
#   U+1006=ဆ   U+1007=ဇ   U+1008=ဈ   U+1009=ည   U+100A=ည
#   U+1010=တ   U+1011=ထ   U+1012=ဒ   U+1013=ဓ   U+1014=န   U+1015=ပ
#   U+1016=ဖ   U+1017=ဗ   U+1018=ဘ   U+1019=မ   U+101A=ယ   U+101B=ရ
#   U+101C=လ   U+101D=ဝ   U+101E=သ   U+101F=ဟ   U+1020=ဠ   U+1021=အ
#   U+102B=ိ   U+102C=ိ (combining vowel marks)
#
# Each tuple is (wrong_char, correct_char) as single-character str.
# substitute_confused_chars uses:  c == wrong_char (str vs str, never int).
CONFUSED_PAIRS = [
    # TA-series (တ ဇ ဒ ဓ): very common real-world confusion
    ('\u1010', '\u1013'), # တ (ta)      -> ဓ (dha)  KEY: fixes ခန္တဝ->ခန္တဝ
    ('\u1013', '\u1010'), # ဓ (dha)    -> တ (ta)
    ('\u1010', '\u1012'), # တ (ta)      -> ဒ (da)
    ('\u1012', '\u1010'), # ဒ (da)      -> တ (ta)
    ('\u1011', '\u1010'), # ထ (tha)     -> တ (ta)
    ('\u1010', '\u1011'), # တ (ta)      -> ထ (tha)
    ('\u1012', '\u1013'), # ဒ (da)      -> ဓ (dha)
    ('\u1013', '\u1012'), # ဓ (dha)     -> ဒ (da)
    # KA-series (က ခ ဂ): visually similar and keyboard-adjacent
    ('\u1000', '\u1001'), # က (ka)      -> ခ (kha)
    ('\u1001', '\u1000'), # ခ (kha)     -> က (ka)
    ('\u1001', '\u1002'), # ခ (kha)     -> ဂ (ga)
    ('\u1002', '\u1001'), # ဂ (ga)      -> ခ (kha)
    # PA-series (ပ ဖ): keyboard-adjacent
    ('\u1015', '\u1016'), # ပ (pa)      -> ဖ (pha)
    ('\u1016', '\u1015'), # ဖ (pha)     -> ပ (pa)
    # NYA confusion (ည ည): extremely common in real Myanmar text
    ('\u1009', '\u100A'), # ည (nya)     -> ည (nnya)
    ('\u100A', '\u1009'), # ည (nnya)    -> ည (nya)
    # NGA / CHA confusion (င ဆ ဈ)
    ('\u1004', '\u1006'), # င (nga)     -> ဆ (cha)
    ('\u1006', '\u1004'), # ဆ (cha)     -> င (nga)
    ('\u1008', '\u1006'), # ဈ (jha)    -> ဆ (cha)
    ('\u1006', '\u1008'), # ဆ (cha)     -> ဈ (jha)
    # YA-group (မ ယ ရ လ ဝ): shape and position confusion
    ('\u1019', '\u101A'), # မ (ma)      -> ယ (ya)
    ('\u101A', '\u1019'), # ယ (ya)      -> မ (ma)
    ('\u101A', '\u101B'), # ယ (ya)      -> ရ (ra)
    ('\u101B', '\u101A'), # ရ (ra)      -> ယ (ya)
    ('\u101B', '\u101C'), # ရ (ra)      -> လ (la)
    ('\u101C', '\u101B'), # လ (la)      -> ရ (ra)
    ('\u101B', '\u101D'), # ရ (ra)      -> ဝ (wa)
    ('\u101D', '\u101B'), # ဝ (wa)      -> ရ (ra)
    # SA/HA confusion (သ ဟ)
    ('\u101E', '\u101F'), # သ (sa)      -> ဟ (ha)

```

```

    ('\u101F', '\u101E'), # ဖ (ha)      -> ဖ (sa)
    # Vowel AA variants
    ('\u102C', '\u102B'), # ဝ (AA)      -> ဝ (tall AA)
    ('\u102B', '\u102C'), # ဝ (tall AA) -> ဝ (AA)
    # Myanmar digit 7 (၇) used as consonant ရ
    ('\u1047', '\u101B'), # ၇ (digit 7) -> ရ (ra)
]

# =====
# Utility Functions
# =====

def _is_base_consonant(c: str) -> bool:
    cp = ord(c)
    return (0x1000 <= cp <= 0x1021) or cp == 0x103F

def _is_myanmar_char(c: str) -> bool:
    cp = ord(c)
    return (0x1000 <= cp <= 0x109F) or cp in (0x200B, 0x200C, 0xFE00, 0xFE01, 0xFEFF)

def _is_myanmar_number_token(token: str) -> bool:
    return bool(token) and all(c in MYANMAR_DIGITS for c in token)

def _is_non_myanmar_token(token: str) -> bool:
    return not token or not any(_is_myanmar_char(c) for c in token)

def _is_passthrough_token(token: str) -> bool:
    return _is_non_myanmar_token(token) or _is_myanmar_number_token(token)

# =====
# Rule-Based Normalizer (updated: per-rule description + detailed logging)
# =====

class BurmeseRuleNormalizer:
    """
    Apply Myanmar-specific regex substitution rules iteratively.

    normalize_detailed() returns per-rule change records so the caller can
    write a fine-grained correction log showing exactly which rule fired.
    """

    # Each rule entry now has 'desc' for log readability.

```

```

RULES = [
    {
        'id': '1',
        'desc': 'strip zero-width/invisible noise chars',
        'pattern': r'[\u200B\u200C\u202C\u00A0\u200D\u200A]',
        'repl': '',
    },
    {
        # Dot-below (U+1037) + Asat (U+103A) ordering fix.
        # Wrong typing: asat THEN dot = U+103A U+1037
        # Correct encoding: dot THEN asat = U+1037 U+103A
        # Explicit Python \u escapes prevent editors from silently reordering.
        'id': '5',
        'desc': 'fix asat(ံ)+dot(့) order -> dot+asat',
        'pattern': '\\u103A\\u1037',          # WRONG order: asat then dot-below
        'repl': '\\u1037\\u103A',          # CORRECT order: dot-below then asat
    },
    {
        # \\1+ catches 2 or more consecutive identical diacritics.
        # (Previous version used \\1{2,} which missed exactly-doubled marks.)
        'id': '15',
        'desc': 'collapse duplicate combining marks',
        'pattern': r'([\\u102B-\\u103E])\\1+',
        'repl': r'\\1',
    },
    {
        'id': '17',
        'desc': '့ံ -> ညံ (wrong independent vowel)',
        'pattern': r'့ံ',
        'repl': 'ညံ',
    },
    {
        'id': '18',
        'desc': '့ော -> ညော (wrong independent vowel)',
        'pattern': r'့ော',
        'repl': 'ညော',
    },
    {
        'id': '20',
        'desc': 'တကသိုလ် -> တက္ကသိုလ် (common misspelling)',
        'pattern': r'တကသိုလ်',
        'repl': 'တက္ကသိုလ်',
    },
    {
        # Rule 25 MUST precede Rule 24: Rule 24 converts ဩ->ဩ first,
        # destroying the longer pattern that Rule 25 needs to match.
        'id': '25',
    }
]

```

```

    'desc': 'ဪ -> ဪ (U+102A, AU independent vowel)',
    'pattern': r'ဪ',
    'repl': 'ဪ',
},
{
    'id': '24',
    'desc': 'ဩ -> ဩ (U+1029, O independent vowel)',
    'pattern': r'ဩ',
    'repl': 'ဩ',
},
{
    'id': '26',
    'desc': 'ငံး -> ငံး (digit 4 used as ငံ)',
    'pattern': r'ငံး',
    'repl': 'ငံး',
},
{
    'id': '28',
    'desc': 'ဈ -> ဈ (two-consonant form to single char)',
    'pattern': r'ဈ',
    'repl': 'ဈ',
},
{
    'id': '29',
    'desc': 'ဏန်း -> ဏန်း (spurious AA vowel)',
    'pattern': r'ဏန်း',
    'repl': 'ဏန်း',
},
{
    'id': '31',
    'desc': 'ဆုင် -> ဆင် (wrong independent vowel)',
    'pattern': r'ဆုင်',
    'repl': 'ဆင်',
},
{
    'id': '33',
    'desc': 'ုိုး -> ဝိုး (vowel U+I reorder)',
    'pattern': r'ုိုး',
    'repl': 'ဝိုး',
},
{
    'id': '34',
    'desc': 'ိုး -> ဝိုး (vowel II+U reorder)',
    'pattern': r'ိုး',
    'repl': 'ဝိုး',
}

```



```

        (rule_id, rule_desc, text_before_rule, text_after_rule)
    """
    all_changes: List[Tuple[str, str, str, str]] = []
    for i in range(max_passes):
        new_text, pass_changes = self._once_detailed(text)
        all_changes.extend(pass_changes)
        if new_text == text:
            return new_text, i + 1, all_changes
        text = new_text
    return text, max_passes, all_changes

# private helpers

def _once(self, text: str) -> str:
    for _rid, _desc, pattern, repl in self._compiled:
        text = pattern.sub(repl, text)
    return text

def _once_detailed(
    self, text: str
) -> Tuple[str, List[Tuple[str, str, str, str]]]:
    changes: List[Tuple[str, str, str, str]] = []
    for rule_id, desc, pattern, repl in self._compiled:
        new_text = pattern.sub(repl, text)
        if new_text != text:
            changes.append((rule_id, desc, text, new_text))
            text = new_text
    return text, changes

# =====
# Syllable Checker (unchanged from v0.5)
# =====

class BurmeseSyllableChecker:
    """Validate Burmese syllables against a frequency dictionary."""

    def __init__(self, dictionary_file: str, min_frequency: int = 2) -> None:
        C = r'[\u1000-\u1021]'
        M = r'(\u103B|\u103C)?\u103D?\u103E?'
        V = (r'\u1031?(?!\u102E\u1030|\u102E\u102F)'
             r'(\u102B|\u102C)?(\u102D\u102F|\u102D|\u102E|\u1032)?'
             r'(\u102F|\u1030)?')
        F = r'[\u1036\u1037]?\u1038?'
        A = r'\u103A'
        S = r'\u1039'
        G = r'\u103B'
        IVS = r'([\u1004])?'

```

```
SUB = r'((?:{C}{S}{C})+)' .format(C=C, S=S)
KZ  = r'\u1019\u1037'
```

```
self.syllable_regex = re.compile(
    f"^(\"
    f\"{IVS}?{C}{M}?{V}?{F}?{A}?|\"
    f\"{IVS}?{SUB}{V}?{F}?{A}?|\"
    f\"{G}{V}?{F}?|\"
    f\"{KZ}{C}{S}{C}{V}?{F}?{A}?|\"
    f\"{C}{S}{C}{V}{F}?{A}?|\"
    f\"{C}{M}{V}{F}?{C}{A}\u1038?|\"
    f\"{C}{SUB}*{V}{C}{A}{F}?|\"
    f\"{C}{KZ}{C}{SUB}{V}?{F}?|\"
    f\"[\u1040-\u1049]+|\"
    f\"[0-9]+|\"
    f\"[\u104A\u104B!?,.<>()\\"'\\-]+|\"
    f\"[a-zA-Z]+\"
    f\")$\",
    re.UNICODE,
)
```

```
self.dictionary: Dict[str, int] = {}
with open(dictionary_file, 'r', encoding='utf-8-sig') as fh:
    for line in fh:
        parts = line.strip().split()
        if len(parts) == 2:
            syllable, freq_str = parts
            syllable = unicodedata.normalize('NFC', syllable)
            freq = int(freq_str)
            if freq >= min_frequency:
                self.dictionary[syllable] = freq
```

```
def in_dictionary(self, syllable: str) -> bool:
    return unicodedata.normalize('NFC', syllable) in self.dictionary
```

```
def matches_re(self, syllable: str) -> bool:
    return bool(self.syllable_regex.fullmatch(syllable))
```

```
def is_valid(self, syllable: str, mode: str = 'dictionary') -> bool:
    if mode == 'RE_and_dictionary':
        return self.matches_re(syllable) and self.in_dictionary(syllable)
    return self.in_dictionary(syllable)
```

```
# =====
# N-gram Language Model (NEW in v0.6)
# =====
```

```

class NgramLanguageModel:
    """
    ARPA format n-gram language model loader with Katz backoff scoring.

    Supports up to trigrams. Used by NgramFuzzyCorrector to rank correction
    candidates by probability in context rather than by simple dictionary
    membership.

    ARPA format (one entry per line, fields whitespace-separated):
        log10prob word [backoff] <- 1-gram
        log10prob word1 word2 [backoff] <- 2-gram
        log10prob word1 word2 word3 <- 3-gram (no backoff)

    Special tokens: <unk>, <s> (begin-of-sentence), </s> (end-of-sentence)
    """

    UNK = '<unk>'
    BOS = '<s>'
    EOS = '</s>'

    # Fallback probability used for any token that is completely absent from
    # the unigram table. -99 is the SRILM convention for "zero probability".
    _DEFAULT_UNK_LOGPROB = -99.0

    def __init__(self, arpa_file: str) -> None:
        # unigrams[word] = (log10_prob, log10_backoff)
        self.unigrams: Dict[str, Tuple[float, float]] = {}
        # bigrams['w1 w2'] = (log10_prob, log10_backoff)
        self.bigrams: Dict[str, Tuple[float, float]] = {}
        # trigrams['w1 w2 w3'] = log10_prob (highest order + no backoff)
        self.trigrams: Dict[str, float] = {}
        self.max_order: int = 1
        self.unk_logprob: float = self._DEFAULT_UNK_LOGPROB

        print(f" Loading n-gram LM: {arpa_file}", file=sys.stderr)
        self._load_arpa(arpa_file)
        # Cache the <unk> probability for fast OOV scoring
        if self.UNK in self.unigrams:
            self.unk_logprob = self.unigrams[self.UNK][0]
        print(
            f" LM loaded: {len(self.unigrams):,} unigrams, "
            f"{len(self.bigrams):,} bigrams, "
            f"{len(self.trigrams):,} trigrams "
            f"(unk={self.unk_logprob:.3f})",
            file=sys.stderr,
        )

    # ARPA loader

```

```

def _load_arpa(self, path: str) -> None:
    current_order = 0
    with open(path, 'r', encoding='utf-8') as fh:
        for raw_line in fh:
            line = raw_line.strip()
            if not line:
                continue
            # Section headers
            # FIX: Changed r'\data\' to '\\data\\' (and same for \end\)
            if line == '\\data\\' or line == '\\end\\':
                continue
            if line.startswith('ngram '):
                # e.g. "ngram 3=6612522"
                try:
                    order = int(line.split('=')[0].split()[1])
                    self.max_order = max(self.max_order, order)
                except (IndexError, ValueError):
                    pass
                continue
            # n-gram section marker: \1-grams: \2-grams: \3-grams:
            # (This regex is fine because the backslash is inside the regex pattern)
            m = re.match(r'\\(\d+)-grams:', line)
            if m:
                current_order = int(m.group(1))
                continue
            if current_order == 0:
                continue

            # Parse n-gram entry
            parts = line.split()
            # Need at least: log_prob + n words
            if len(parts) < current_order + 1:
                continue
            try:
                log_prob = float(parts[0])
            except ValueError:
                continue

            words = parts[1: current_order + 1] # the n words
            # Optional backoff weight (present for orders < max_order)
            backoff = 0.0
            if len(parts) > current_order + 1:
                try:
                    backoff = float(parts[current_order + 1])
                except ValueError:
                    backoff = 0.0

```

```

        key = ' '.join(words)    # space-joined; Myanmar words have no spaces
        if current_order == 1:
            self.unigrams[key] = (log_prob, backoff)
        elif current_order == 2:
            self.bigrams[key] = (log_prob, backoff)
        elif current_order == 3:
            self.trigrams[key] = log_prob

# Scoring

def score_word(self, word: str, context: List[str]) -> float:
    """
    Return log10 P(word | context) using Katz backoff.

    context : list of preceding words, most-recent last.
               Caller passes out_tokens[-2:] so context length is 0-2.

    The returned value is always 0 (log10 of a probability).
    Lower (more negative) = less likely.
    """
    w = word
    ctx = context    # e.g. [prev2, prev1]

    # Try trigram P(w | prev2, prev1)
    if len(ctx) >= 2 and self.max_order >= 3:
        key3 = f"{ctx[-2]} {ctx[-1]} {w}"
        if key3 in self.trigrams:
            return self.trigrams[key3]
        # Not found → apply bigram backoff and fall through to bigram
        bow2_key = f"{ctx[-2]} {ctx[-1]}"
        bow2 = self.bigrams.get(bow2_key, (0.0, 0.0))[1]    # log backoff
        return bow2 + self._score_bigram(w, ctx[-1])

    # Try bigram P(w | prev1)
    if len(ctx) >= 1:
        return self._score_bigram(w, ctx[-1])

    # Unigram only
    return self._score_unigram(w)

def _score_bigram(self, word: str, prev: str) -> float:
    key2 = f"{prev} {word}"
    if key2 in self.bigrams:
        return self.bigrams[key2][0]
    # Backoff: bow(prev) + P(word)
    bow1 = self.unigrams.get(prev, (0.0, 0.0))[1]
    return bow1 + self._score_unigram(word)

```

```

def _score_unigram(self, word: str) -> float:
    if word in self.unigrams:
        return self.unigrams[word][0]
    return self.unk_logprob

# =====
# N-gram-based Fuzzy Corrector (NEW in v0.6)
# =====

class NgramFuzzyCorrector:
    """
    Fuzzy corrector that uses an n-gram language model for syllable-level
    confirmation and substitution.

    For each unknown token the corrector:
    1. Generates correction candidates (same four methods as
       TargetedFuzzyCorrector: dup-vowel removal, asat removal,
       confused-char substitution, medial ordering fix).
    2. FILTERS candidates to keep ONLY those that exist in the syllable
       dictionary. (This prevents blind character swaps on OOV syllables
       based purely on backoff noise).
    3. Scores the ORIGINAL token and remaining valid candidates with the
       LM in the current left-context window.
    4. Accepts the highest-scoring candidate ONLY IF it beats the original
       token's score by at least `min_improve` log10 units.
    """

    def __init__(
        self,
        lm: NgramLanguageModel,
        dictionary: Dict[str, int],
        min_improve: float = 0.5,
    ) -> None:
        self.lm = lm
        self.dictionary = dictionary
        self.min_improve = min_improve
        # Cache keyed by (token, context_tuple) to avoid repeated LM lookups
        self._cache: Dict[Tuple[str, ...], Optional[Tuple[str, str]]] = {}

    # Candidate generators (same logic as TargetedFuzzyCorrector)

    def _gen_dup_vowel(self, token: str) -> List[str]:
        chars = list(token)
        lo, hi = COMBINING_MARKS_RANGE
        for i in range(len(chars) - 1):
            if chars[i] == chars[i + 1] and lo <= ord(chars[i]) <= hi:
                return [''.join(chars[:i] + chars[i + 1:])]

```

```

    return []

def _gen_asat_removal(self, token: str) -> List[str]:
    results, tone_marks = [], {'\u1037', '\u1038'}
    idx = token.find(ASAT)
    while idx != -1:
        if idx + 1 < len(token) and token[idx + 1] in tone_marks:
            results.append(token[:idx] + token[idx + 1:])
            idx = token.find(ASAT, idx + 1)
    return results

def _gen_char_sub(self, token: str) -> List[str]:
    candidates, seen = [], set()
    chars = list(token)
    for i, c in enumerate(chars):
        for wrong_char, correct_char in CONFUSED_PAIRS:
            if c == wrong_char:
                cand = ''.join(chars[:i] + [correct_char] + chars[i + 1:])
                if cand not in seen:
                    candidates.append(cand)
                    seen.add(cand)
    return candidates

def _gen_medial(self, token: str) -> List[str]:
    candidates = []
    for medial in ('\u103B', '\u103C', '\u103D'):
        idx = token.find(medial)
        if idx != -1 and idx + 1 < len(token):
            nxt = token[idx + 1]
            ncp = ord(nxt)
            if (0x1000 <= ncp <= 0x1021) or ncp == 0x103F:
                cand = token.replace(medial + nxt, nxt + medial, 1)
                if cand != token:
                    candidates.append(cand)
    return candidates

# Main correction entry point

def correct(
    self, token: str, context: List[str]
) -> Optional[Tuple[str, str]]:
    """
    Return (corrected_token, method_name) or None.
    """
    cache_key = (token, *context[-2:])
    if cache_key in self._cache:
        return self._cache[cache_key]

```

```

# Score the original token in context
orig_score = self.lm.score_word(
    unicodedata.normalize('NFC', token), context
)

# Collect all (candidate, method) pairs
all_candidates: List[Tuple[str, str]] = []
for c in self._gen_dup_vowel(token):
    all_candidates.append((c, 'ngram_dup_vowel'))
for c in self._gen_asat_removal(token):
    all_candidates.append((c, 'ngram_asat_removal'))
for c in self._gen_char_sub(token):
    all_candidates.append((c, 'ngram_char_sub'))
for c in self._gen_medial(token):
    all_candidates.append((c, 'ngram_medial'))
# Combined: dup-vowel then asat removal
for dc in self._gen_dup_vowel(token):
    for ac in self._gen_asat_removal(dc):
        all_candidates.append((ac, 'ngram_dup_plus_asat'))

# Score every candidate; keep the best that beats orig by min_improve
best_result: Optional[Tuple[str, str]] = None
best_score = orig_score + self.min_improve # threshold

for cand, method in all_candidates:
    cand_nfc = unicodedata.normalize('NFC', cand)

    # =====
    # CRITICAL FIX: Syllable-level confirmation
    # Only evaluate candidates that are VALID syllables in the
    # dictionary. The LM is for contextual confirmation between
    # valid syllables, NOT for inventing new syllables via blind
    # character swaps on OOV tokens.
    # =====
    if cand_nfc not in self.dictionary:
        continue

    score = self.lm.score_word(cand_nfc, context)
    if score > best_score:
        best_score = score
        best_result = (cand_nfc, method)

self._cache[cache_key] = best_result
return best_result

# =====
# Dictionary-based Fuzzy Corrector (unchanged from v0.5, kept for compat)
# =====

```

```

class TargetedFuzzyCorrector:
    """
    Targeted fuzzy correction against the syllable frequency dictionary.

    Used when --ngram-lm is NOT provided and --fuzzy-distance 1 is set.
    Preserved for backward compatibility with v0.5 behaviour.
    """

    def __init__(self, dictionary: Dict[str, int]) -> None:
        self.dictionary = dictionary
        self._cache: Dict[str, Optional[Tuple[str, str]]] = {}

    def _in_dict(self, candidate: str) -> bool:
        return unicodedata.normalize('NFC', candidate) in self.dictionary

    def _dup_vowel(self, token: str) -> List[str]:
        chars = list(token)
        lo, hi = COMBINING_MARKS_RANGE
        for i in range(len(chars) - 1):
            if chars[i] == chars[i + 1] and lo <= ord(chars[i]) <= hi:
                return [''.join(chars[:i] + chars[i + 1:])]
        return []

    def _asat_removal(self, token: str) -> List[str]:
        results, tone_marks = [], {'\u1037', '\u1038'}
        idx = token.find(ASAT)
        while idx != -1:
            if idx + 1 < len(token) and token[idx + 1] in tone_marks:
                results.append(token[:idx] + token[idx + 1:])
            idx = token.find(ASAT, idx + 1)
        return results

    def _char_sub(self, token: str) -> List[str]:
        candidates, seen = [], set()
        chars = list(token)
        for i, c in enumerate(chars):
            for wrong_char, correct_char in CONFUSED_PAIRS:
                if c == wrong_char:
                    cand = ''.join(chars[:i] + [correct_char] + chars[i + 1:])
                    if cand not in seen:
                        candidates.append(cand)
                        seen.add(cand)
        return candidates

    def _medial(self, token: str) -> List[str]:
        candidates = []
        for medial in ('\u103B', '\u103C', '\u103D'):

```

```

        idx = token.find(medial)
        if idx != -1 and idx + 1 < len(token):
            nxt = token[idx + 1]
            ncp = ord(nxt)
            if (0x1000 <= ncp <= 0x1021) or ncp == 0x103F:
                cand = token.replace(medial + nxt, nxt + medial, 1)
                if cand != token:
                    candidates.append(cand)
    return candidates

def correct(self, token: str) -> Optional[Tuple[str, str]]:
    if token in self._cache:
        return self._cache[token]

    result = None
    for cand in self._dup_vowel(token):
        if self._in_dict(cand):
            result = (cand, 'duplicate_vowel_removal'); break
    if result is None:
        for cand in self._asat_removal(token):
            if self._in_dict(cand):
                result = (cand, 'asat_removal'); break
    if result is None:
        for cand in self._char_sub(token):
            if self._in_dict(cand):
                result = (cand, 'character_substitution'); break
    if result is None:
        for cand in self._medial(token):
            if self._in_dict(cand):
                result = (cand, 'medial_ordering'); break
    if result is None:
        for dc in self._dup_vowel(token):
            for ac in self._asat_removal(dc):
                if self._in_dict(ac):
                    result = (ac, 'duplicate_vowel_plus_asat_removal')
                    break
            if result:
                break

    self._cache[token] = result
    return result

# =====
# Consonant+Asat Merger (unchanged from v0.5)
# =====

def is_consonant_with_athat(token: str) -> bool:

```

```

    if not token:
        return False
    first_cp = ord(token[0])
    if not ((0x1000 <= first_cp <= 0x1021) or first_cp == 0x103F
            or first_cp == 0x1039):
        return False
    asat_pos = token.find(ASAT)
    if asat_pos == -1:
        return False
    for c in token[asat_pos + 1:]:
        if ord(c) not in (0x1037, 0x1038):
            return False
    return True

def try_merge_with_previous(
    current_token: str,
    previous_token: str,
    dictionary: Dict[str, int],
) -> Optional[str]:
    if not is_consonant_with_athat(current_token):
        return None
    merged = unicodedata.normalize('NFC', previous_token + current_token)
    return merged if merged in dictionary else None

# =====
# Compound-Split Recovery (unchanged from v0.5)
# =====

def compound_split(
    token: str,
    dictionary: Dict[str, int],
    max_parts: int = 3,
) -> Optional[List[str]]:
    n = len(token)
    dp: List[Optional[List[str]]] = [None] * (n + 1)
    dp[0] = []
    for i in range(1, n + 1):
        for j in range(max(0, i - 12), i):
            if dp[j] is None or len(dp[j]) >= max_parts:
                continue
            part_nfc = unicodedata.normalize('NFC', token[j:i])
            if part_nfc in dictionary:
                cand = dp[j] + [part_nfc]
                if dp[i] is None or len(cand) < len(dp[i]):
                    dp[i] = cand
    result = dp[n]

```

```

    return result if (result is not None and len(result) >= 2) else None

# =====
# Main Processing Pipeline
# =====

def process_input(
    input_stream,
    output_file: Optional[str] = None,
    error_output_file: Optional[str] = 'detected_syllable_errors.txt',
    verbose: bool = False,
    dictionary_file: Optional[str] = None,
    min_frequency: int = 2,
    check: str = 'dictionary',
    fuzzy_distance: int = 0,
    ngram_lm_file: Optional[str] = None,
    min_lm_improve: float = 0.5,
    log_file: Optional[str] = None,
    debug_fuzzy: bool = False,
) -> None:
    """
    Full normalisation pipeline.

    Token processing order
    -----
    0. Passthrough (Latin / Myanmar digit-only tokens) + copy unchanged.
    1. Unicode NFC normalization.
    2. Stage 1: iterative regex rules; log each rule that fires.
    3. Dictionary validity check + done if valid.
    4. Stage 2: fuzzy correction (n-gram LM if --ngram-lm, else syllable-dict
       if --fuzzy-distance 1, else skipped).
    5. Stage 3: consonant+asat merge with previous output token.
    6. Stage 4: compound split (DP, 3 parts, len 4).
    7. Unknown: output as-is; counted in stats.
    """

    # Initialise components
    checker = BurmeseSyllableChecker(dictionary_file, min_frequency)
    rule_norm = BurmeseRuleNormalizer()

    # Determine which fuzzy corrector to use
    ngram_corrector: Optional[NgramFuzzyCorrector] = None
    target_corrector: Optional[TargetedFuzzyCorrector] = None

    if ngram_lm_file:
        lm = NgramLanguageModel(ngram_lm_file)
        ngram_corrector = NgramFuzzyCorrector(lm, checker.dictionary, min_improve=min_lm_improve)

```

```

print(
    f" Fuzzy correction: n-gram LM mode "
    f"(min_improve={min_lm_improve:.2f} log10 units)",
    file=sys.stderr,
)
elif fuzzy_distance > 0:
    target_corrector = TargetedFuzzyCorrector(checker.dictionary)
    print(
        " Fuzzy correction: syllable-dictionary mode (v0.5 compatible)",
        file=sys.stderr,
    )
else:
    print(" Fuzzy correction: disabled", file=sys.stderr)

# Stats counters
stats: Dict[str, int] = {
    'lines': 0,
    'tokens': 0,
    'passthrough': 0,
    'already_valid': 0,
    'fixed_stage1': 0, # tokens changed by rules
    'fixed_stage1_rule_apps': 0, # individual rule applications
    'fixed_ngram_dup_vowel': 0,
    'fixed_ngram_asat': 0,
    'fixed_ngram_char_sub': 0,
    'fixed_ngram_medial': 0,
    'fixed_ngram_combined': 0,
    'fixed_fuzzy_dup_vowel': 0, # dict-based (v0.5 compat)
    'fixed_fuzzy_asat': 0,
    'fixed_fuzzy_char_sub': 0,
    'fixed_fuzzy_medial': 0,
    'fixed_fuzzy_combined': 0,
    'fixed_merge': 0,
    'fixed_split': 0,
    'still_unknown': 0,
}

log_entries: List[str] = []
error_output_lines: List[str] = []

def _log(line_no: int, original: str, corrected: str, stage: str) -> None:
    if log_file or verbose:
        log_entries.append(
            f"line {line_no:>6} | {original!r:30s}"
            f"-> {corrected!r:30s} | {stage}"
        )

output_lines: List[str] = []

```

```

# Main loop
for raw_line in input_stream:
    line = raw_line.rstrip('\n')
    stats['lines'] += 1
    line_no = stats['lines']

    tokens = line.split()
    out_tokens: List[str] = []
    error_tokens: List[str] = []

    for tok in tokens:
        stats['tokens'] += 1

        # Stage 0: passthrough
        if _is_passthrough_token(tok):
            stats['passthrough'] += 1
            out_tokens.append(tok)
            error_tokens.append(tok)
            continue

        # Stage 0b: NFC normalisation
        tok_nfc = unicodedata.normalize('NFC', tok)

        # Stage 1: regex rules with per-rule logging
        tok_norm, _passes, rule_changes = rule_norm.normalize_detailed(tok_nfc)

        # Log every individual rule that fired (not just "rules applied")
        for rule_id, rule_desc, before, after in rule_changes:
            _log(line_no, before, after,
                f'stage1:rule{rule_id} ({rule_desc})')
            stats['fixed_stage1_rule_apps'] += 1

        if tok_norm != tok_nfc:
            stats['fixed_stage1'] += 1
            tok = tok_norm
        else:
            tok = tok_nfc # ensure NFC even if no rules fired

        # Dictionary validity check
        if checker.is_valid(tok, mode=check):
            stats['already_valid'] += 1
            out_tokens.append(tok)
            error_tokens.append(tok)
            continue

        # Mark as error BEFORE attempting correction
        error_tokens.append(f'<{tok}>')

```

```

# Stage 2a: N-gram LM fuzzy correction
if ngram_corrector is not None:
    context = out_tokens[-2:] # up to 2 preceding output tokens
    ng_result = ngram_corrector.correct(tok, context)
    if ng_result is not None:
        corrected, method = ng_result
        _log(line_no, tok, corrected, f'stage2:ngram({method})')
        if 'dup_plus_asat' in method: stats['fixed_ngram_combined'] += 1
        elif 'dup_vowel' in method: stats['fixed_ngram_dup_vowel'] += 1
        elif 'asat' in method: stats['fixed_ngram_asat'] += 1
        elif 'char_sub' in method: stats['fixed_ngram_char_sub'] += 1
        elif 'medial' in method: stats['fixed_ngram_medial'] += 1
        out_tokens.append(corrected)
        continue

# Stage 2b: Dict-based fuzzy correction (v0.5 compat)
if target_corrector is not None:
    t_result = target_corrector.correct(tok)
    if t_result is not None:
        corrected, method = t_result
        _log(line_no, tok, corrected, f'stage2:fuzzy({method})')
        if 'plus_asat' in method: stats['fixed_fuzzy_combined'] += 1
        elif 'duplicate' in method: stats['fixed_fuzzy_dup_vowel'] += 1
        elif 'asat' in method: stats['fixed_fuzzy_asat'] += 1
        elif 'character_sub' in method: stats['fixed_fuzzy_char_sub'] += 1
        elif 'medial' in method: stats['fixed_fuzzy_medial'] += 1
        out_tokens.append(corrected)
        continue

# Stage 3: consonant+asat merge with previous token
if out_tokens and is_consonant_with_athat(tok):
    prev = out_tokens[-1]
    merged = try_merge_with_previous(tok, prev, checker.dictionary)
    if merged:
        _log(line_no, f"{prev} {tok}", merged,
             'stage3:merge_with_previous')
        stats['fixed_merge'] += 1
        out_tokens[-1] = merged
        continue

# Stage 4: compound split
if len(tok) >= 4:
    parts = compound_split(tok, checker.dictionary)
    if parts is not None:
        _log(line_no, tok, ' '.join(parts),
             f'stage4:split({len(parts)} parts)')
        stats['fixed_split'] += 1

```

```

        out_tokens.extend(parts)
        continue

    # Still unknown
    stats['still_unknown'] += 1
    out_tokens.append(tok)

    output_lines.append(' '.join(out_tokens))
    error_output_lines.append(' '.join(error_tokens))

# Write outputs
if output_file:
    with open(output_file, 'w', encoding='utf-8') as fh:
        for line in output_lines:
            fh.write(line.strip() + '\n')
else:
    for line in output_lines:
        print(line.strip())

if error_output_file:
    with open(error_output_file, 'w', encoding='utf-8') as fh:
        for line in error_output_lines:
            fh.write(line.strip() + '\n')

if log_file and log_entries:
    with open(log_file, 'w', encoding='utf-8') as fh:
        fh.write('\n'.join(log_entries) + '\n')

# Summary statistics
total_ngram = (stats['fixed_ngram_dup_vowel'] + stats['fixed_ngram_asat']
               + stats['fixed_ngram_char_sub'] + stats['fixed_ngram_medial']
               + stats['fixed_ngram_combined'])
total_fuzzy = (stats['fixed_fuzzy_dup_vowel'] + stats['fixed_fuzzy_asat']
               + stats['fixed_fuzzy_char_sub'] + stats['fixed_fuzzy_medial']
               + stats['fixed_fuzzy_combined'])
total_fixed = (stats['fixed_stage1'] + total_ngram + total_fuzzy
               + stats['fixed_merge'] + stats['fixed_split'])
total_tokens = max(1, stats['tokens'])

print(
    f"\n=== syl_normalizer summary (v0.6) ===\n"
    f" Lines processed      : {stats['lines']:>10,}\n"
    f" Tokens processed      : {stats['tokens']:>10,}\n"
    f" Passthrough           : {stats['passthrough']:>10,}\n"
    f" Already valid         : {stats['already_valid']:>10,}\n"
    f"\n"
    f" Fixed - stage 1 (rules, token count)      : {stats['fixed_stage1']:>8,}\n"
    f" Fixed - stage 1 (individual rule apps)    : {stats['fixed_stage1_rule_apps']:>8,}\n"

```

```

f"\n"
f" Fixed - stage 2 ngram (dup vowel)      : {stats['fixed_ngram_dup_vowel']:>8,}\n"
f" Fixed - stage 2 ngram (asat removal)    : {stats['fixed_ngram_asat']:>8,}\n"
f" Fixed - stage 2 ngram (char sub)        : {stats['fixed_ngram_char_sub']:>8,}\n"
f" Fixed - stage 2 ngram (medial order)    : {stats['fixed_ngram_medial']:>8,}\n"
f" Fixed - stage 2 ngram (dup+asat)        : {stats['fixed_ngram_combined']:>8,}\n"
f"\n"
f" Fixed - stage 2 dict (dup vowel)        : {stats['fixed_fuzzy_dup_vowel']:>8,}\n"
f" Fixed - stage 2 dict (asat removal)      : {stats['fixed_fuzzy_asat']:>8,}\n"
f" Fixed - stage 2 dict (char sub)         : {stats['fixed_fuzzy_char_sub']:>8,}\n"
f" Fixed - stage 2 dict (medial order)     : {stats['fixed_fuzzy_medial']:>8,}\n"
f" Fixed - stage 2 dict (dup+asat)         : {stats['fixed_fuzzy_combined']:>8,}\n"
f"\n"
f" Fixed - stage 3 (merge with previous)    : {stats['fixed_merge']:>8,}\n"
f" Fixed - stage 4 (compound split)        : {stats['fixed_split']:>8,}\n"
f"\n"
f" Total fixed          : {total_fixed:>10,}\n"
f" Still unknown       : {stats['still_unknown']:>10,}\n"
f" Unknown rate        : {stats['still_unknown']/total_tokens:>9.2%}\n",
file=sys.stderr,
)

# =====
# CLI Entry Point
# =====

def main() -> None:
    parser = argparse.ArgumentParser(
        description=(
            'Burmese Syllable Normalizer v0.6 - '
            'Normalizes Myanmar text using rules, n-gram LM fuzzy correction, '
            'consonant+asat merging, and compound splitting.'
        ),
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=__doc__,
    )

    # I/O arguments
    parser.add_argument('--input', '-i',
        help='Input file (space-separated syllables). Reads from stdin if omitted.')
    parser.add_argument('--output', '-o',
        help='Output file with corrected text. Prints to stdout if omitted.')
    parser.add_argument('--log', '-l',
        help='File to write detailed per-correction log (every rule + every fix).')
    parser.add_argument('--error-output', '-e',
        default='detected_syllable_errors.txt',
        help='File to write detected errors with <markers> BEFORE correction.')

```

```

parser.add_argument('--verbose', '-v', action='store_true',
                    help='Also log unknown tokens to the correction log.')

# Dictionary arguments
parser.add_argument('--dictionary', '-d', required=True,
                    help='Syllable dictionary (format: syllable<SPACE>frequency, one per line).')
parser.add_argument('--frequency', '-f', type=int, default=2, metavar='N',
                    help='Minimum frequency for a dictionary syllable to be considered valid '
                         '(default: 2).')
parser.add_argument('--check', '-c',
                    choices=['dictionary', 'RE_and_dictionary'], default='dictionary',
                    help='Syllable validation mode (default: dictionary).')

# Fuzzy correction arguments
parser.add_argument('--fuzzy-distance', '-z', type=int, default=0, metavar='N',
                    dest='fuzzy_distance',
                    help='Enable syllable-dictionary fuzzy correction (0=disabled, 1=enabled). '
                         'Ignored when --ngram-lm is provided.')
parser.add_argument('--ngram-lm', metavar='ARPA_FILE',
                    dest='ngram_lm_file', default=None,
                    help='ARPA format n-gram language model for smarter fuzzy correction. '
                         'When provided, overrides --fuzzy-distance and uses LM-based scoring.')
parser.add_argument('--min-lm-improve', type=float, default=0.5,
                    dest='min_lm_improve', metavar='DELTA',
                    help='Minimum log10 probability improvement required for an n-gram '
                         'correction to be accepted (default: 0.5). Higher = more conservative.')

# Debug arguments
parser.add_argument('--debug-fuzzy', action='store_true',
                    help='Print verbose fuzzy correction debug output to stderr.')

args = parser.parse_args()

input_stream = (
    open(args.input, 'r', encoding='utf-8') if args.input else sys.stdin
)

try:
    process_input(
        input_stream      = input_stream,
        output_file       = args.output,
        error_output_file = args.error_output if args.error_output else None,
        verbose           = args.verbose,
        dictionary_file    = args.dictionary,
        min_frequency      = args.frequency,
        check              = args.check,
        fuzzy_distance     = args.fuzzy_distance,
        ngram_lm_file      = args.ngram_lm_file,
    )

```

```

        min_lm_improve    = args.min_lm_improve,
        log_file          = args.log,
        debug_fuzzy       = args.debug_fuzzy,
    )
    finally:
        if args.input:
            input_stream.close()

if __name__ == '__main__':
    main()

```

1.4 Shell Script for Syllable Normalization

Training/Dev/Test ဒေတာ အားလုံးကို normalization လုပ်ဖို့အတွက် shell တစ်ပုဒ်ရေးထားတယ်။

```
[4]: %cd /home/ye/exp/SMT-NMT_tutorial/clean-data/big/normalize
```

```
/home/ye/exp/SMT-NMT_tutorial/clean-data/big/normalize
```

```
[5]: !cat ./normalize_all.sh
```

```
#!/bin/bash
```

```
# Stage 1 + merge/split only (safest)
```

```
time python3 /home/ye/exp/SMT-NMT_tutorial/syl_normalizer/syl_normalizer.py \
--dictionary /home/ye/exp/SMT-
NMT_tutorial/syl_normalizer/final_syl_dictionary_13Feb2024.sorted.txt \
--frequency 2 \
--input ./train.my --output ./train.my.normalize \
--log corrections_train.log --error-output errors_train.txt \
--fuzzy-distance 0
```

```
time python3 /home/ye/exp/SMT-NMT_tutorial/syl_normalizer/syl_normalizer.py \
--dictionary /home/ye/exp/SMT-
NMT_tutorial/syl_normalizer/final_syl_dictionary_13Feb2024.sorted.txt \
--frequency 2 \
--input ./dev.my --output ./dev.my.normalize \
--log corrections_dev.log --error-output errors_dev.txt \
--fuzzy-distance 0
```

```
time python3 /home/ye/exp/SMT-NMT_tutorial/syl_normalizer/syl_normalizer.py \
--dictionary /home/ye/exp/SMT-
NMT_tutorial/syl_normalizer/final_syl_dictionary_13Feb2024.sorted.txt \
--frequency 2 \
--input ./test.my --output ./test.my.normalize \
--log corrections_test.log --error-output errors_test.txt \
--fuzzy-distance 0
```

```
[6]: !./normalize_all.sh
```

Fuzzy correction: disabled

=== syl_normalizer summary (v0.6) ===

```
Lines processed      :    62,625
Tokens processed     :   734,667
Passthrough         :    15,461
Already valid        :   704,878
```

```
Fixed - stage 1 (rules, token count) :      82
Fixed - stage 1 (individual rule apps) :      82
```

```
Fixed - stage 2 ngram (dup vowel)     :      0
Fixed - stage 2 ngram (asat removal)   :      0
Fixed - stage 2 ngram (char sub)       :      0
Fixed - stage 2 ngram (medial order)   :      0
Fixed - stage 2 ngram (dup+asat)       :      0
```

```
Fixed - stage 2 dict (dup vowel)       :      0
Fixed - stage 2 dict (asat removal)    :      0
Fixed - stage 2 dict (char sub)        :      0
Fixed - stage 2 dict (medial order)    :      0
Fixed - stage 2 dict (dup+asat)        :      0
```

```
Fixed - stage 3 (merge with previous) :  12,876
Fixed - stage 4 (compound split)       :      19
```

```
Total fixed         :    12,977
Still unknown        :     1,433
Unknown rate         :     0.20%
```

real 0m2.273s

user 0m2.237s

sys 0m0.032s

Fuzzy correction: disabled

=== syl_normalizer summary (v0.6) ===

```
Lines processed      :    10,000
Tokens processed     :   117,871
Passthrough         :     2,648
Already valid        :   112,909
```

```
Fixed - stage 1 (rules, token count) :      13
Fixed - stage 1 (individual rule apps) :      13
```

```
Fixed - stage 2 ngram (dup vowel)     :      0
```

```

Fixed - stage 2 ngram (asat removal)      :      0
Fixed - stage 2 ngram (char sub)           :      0
Fixed - stage 2 ngram (medial order)       :      0
Fixed - stage 2 ngram (dup+asat)          :      0

Fixed - stage 2 dict (dup vowel)           :      0
Fixed - stage 2 dict (asat removal)        :      0
Fixed - stage 2 dict (char sub)            :      0
Fixed - stage 2 dict (medial order)        :      0
Fixed - stage 2 dict (dup+asat)            :      0

Fixed - stage 3 (merge with previous)      :    2,064
Fixed - stage 4 (compound split)           :         3

Total fixed          :    2,080
Still unknown        :      247
Unknown rate         :    0.21%

```

```

real    0m0.385s
user    0m0.374s
sys      0m0.011s
  Fuzzy correction: disabled

```

=== syl_normalizer summary (v0.6) ===

```

Lines processed      :    8,000
Tokens processed     :   93,908
Passthrough         :    2,046
Already valid        :   89,991

```

```

Fixed - stage 1 (rules, token count)      :    10
Fixed - stage 1 (individual rule apps)     :    10

Fixed - stage 2 ngram (dup vowel)          :      0
Fixed - stage 2 ngram (asat removal)       :      0
Fixed - stage 2 ngram (char sub)           :      0
Fixed - stage 2 ngram (medial order)       :      0
Fixed - stage 2 ngram (dup+asat)          :      0

Fixed - stage 2 dict (dup vowel)           :      0
Fixed - stage 2 dict (asat removal)        :      0
Fixed - stage 2 dict (char sub)            :      0
Fixed - stage 2 dict (medial order)        :      0
Fixed - stage 2 dict (dup+asat)            :      0

Fixed - stage 3 (merge with previous)      :    1,678
Fixed - stage 4 (compound split)           :         1

```

```
Total fixed      :      1,689
Still unknown    :        192
Unknown rate     :      0.20%
```

```
real    0m0.316s
user    0m0.305s
sys     0m0.011s
```

1.5 Check Normalized Outputs

Normalization လုပ်သွားတဲ့ အပိုင်းကို အသေးစိတ် လေ့လာကြည့်ကြရအောင်။

```
[7]: !head train.my.normalize
```

```
အောက် ပါ ရော ဂါ လက္ခဏာ တွေ များ ရှိ ရင်
စိတ် ကို အေး အေး ဆေး ဆေး ထား ပါ
နာ တာ ရှည် ရော ဂါ ရယ် လို့ မ ရှိ ပါ ဘူး ဗျ
လက္ခဏာ တွေ နဲ့
ပြီး တဲ့ အ ခါ နား ကြပ် ကို တံ တောင့် ဆစ် နား မှာ ထား မှာ မို့ ပါ ရှင်
ပု စွန် တော့ ကျွန် တော် စား တယ်
သို့ ပေ မဲ့ လည်း
ကျွန် တော် ခေါင်း ပေါ် ရေ ခဲ တင် ထား ရ မ လား
ကွန် ပြူ တာ ရောင် ခြည် နဲ့ ထပ် စစ် ဆေး ပါ မယ် နော်
ဒီ ကြား မှာ ခ ဣ လေး ထိုင် စောင့် ပေး လို့ ရ မ လား
```

Normalization မလုပ်ခင်က ဖိုင်နဲ့ နှိုင်းယှဉ်ကြည့်ရအောင်

```
[8]: !head train.my
```

```
အောက် ပါ ရော ဂါ လက္ခဏာ တွေ များ ရှိ ရင်
စိတ် ကို အေး အေး ဆေး ဆေး ထား ပါ
နာ တာ ရှည် ရော ဂါ ရယ် လို့ မ ရှိ ပါ ဘူး ဗျ
လက္ခဏာ တွေ နဲ့
ပြီး တဲ့ အ ခါ နား ကြပ် ကို တံ တော င် ဆစ် နား မှာ ထား မှာ မို့ ပါ ရှင်
ပု စွန် တော့ ကျွန် တော် စား တယ်
သို့ ပေ မဲ့ လည်း
ကျွန် တော် ခေါင်း ပေါ် ရေ ခဲ တင် ထား ရ မ လား
ကွန် ပြူ တာ ရောင် ခြည် နဲ့ ထပ် စစ် ဆေး ပါ မယ် နော်
ဒီ ကြား မှာ ခ ဣ လေး ထိုင် စော င် ပေး လို့ ရ မ လား
```

Error Detection လုပ်ထားတဲ့အပိုင်း တချို့ကိုပဲ ဆွဲထုတ်ကြည့်ရအောင်။

```
[13]: !grep -n "<" ./errors_train.txt | head -n 30
```

```
5:ပြီး တဲ့ အ ခါ နား ကြပ် ကို တံ တော <င> ဆစ် နား မှာ ထား မှာ မို့ ပါ ရှင်
10:ဒီ ကြား မှာ ခ ဣ လေး ထိုင် စော <င> ပေး လို့ ရ မ လား
```

14: တို့ တွေ အ နည်း ဆုံး သွား ကို ၃ - ၅ မိ နစ် အ ထိ အ နည်း ဆုံး တိုက် သ <ငံ> ┘
တယ်

၅၂ <ငံ>

20: ဘယ် အ ချိန် က တည်း က သင် ၏ မျက် လုံး တွေ များ သည် ရဲ ရဲ နီ နေ ပြီး အ ┘
ရည် တွေ

နဲ့ ပြ <ည့်> နှက် နေ သ နည်း

21: သား အိမ် ခေါင်း ပွ <ငံ> တာ ကို သုံး သပ် ချင် လို့ ပါ

24: ငါ အ ပူ ချိန် မ တိုင်း ရ သေး ပေ မ <ယံ> ကိုယ် ပူ နေ တယ် လို့ ခံ စား ရ တယ်

29: လူ နာ ရှင် <ကိုး> တာ ခေါ် ထား ပါ တယ် ခင် ဗျာ လာ ပါ လိ <မ့်> မယ်

34: ဘာ လုပ် သ <ငံ> တယ် ဆို တာ ပြော ဖို့ စော လွန်း နေ ပါ သေး တယ် Neuro p ┘

၀ s y

ch i a t r y ဆို တာ က m e t a p h y s i c a l ဆေး တစ် မျိုး ပါ အ ခု အ ချိန် ┘
က စ

ပြီး အ တူ တူ ကျော် ဖြတ် ကြ ရ အောင်

35: သ <ငံ> အ နေ နဲ့ ဖုန်း နဲ့ မ ဟုတ် ဘဲ အ ခြား ဘယ် လို နည်း လမ်း မျိုး နဲ့ ဆက်
သွယ် စေ ချင် ပါ သ လဲ

40: ပုံ မှန် အား ဖြ <ငံ> ကိုယ် တို့ ရဲ့ ကိုယ် အ ပူ ချိန် က ၃ ၇ . ၅ ဒီ ဂ ရီ ထက် မ
ကျော် ရ ဘူး

45: လက် ထိပ် မှာ အောက် ဆီ ဂျင် ပြ <ည့်> ၀ မှု သည် 9 6 % ရှိ သည်

46: ရော ဂါ လက္ခဏာ က <ရှေး> ဖြစ် နေ တာ နဲ့ တူ တယ်

47: ကိုယ် အား ဆ <နံ> ပြီး ငြိမ် ငြိမ် နေ ပေး ပါ ရှင်

52: အ ပြင် မှာ စော <ငံ> နေ ပေး လို့ ရ နိုင် မ လား

55: ဆ ရာ မ လေး ၅၂ <ငံ>

58: ဓာတ် မ တ <ည့်> တာ က တစ် ဆ <ငံ> သ တိ လစ် တာ မျိုး ဖြစ် နိုင် ပါ တယ်

61: ဆ ရာ က လူ နာ ရဲ့ ဗိုက် ကို ဖိ နှိပ် ကြ <ည့်> ပါ မယ်

65: အ ဖျား ရှိ နေ တယ် နော် ၅၂ <ငံ>

68: တော် တော် ကို ထိတ် လ <နံ> ပြီး အော် ဟစ် အ ကူ အ ညီ ရှာ ပေ မ <ယံ> ရပ် ကြ
<ည့်> နေ ခဲ့ ကြ တယ်

72: အောက် ကို ငုံ့ ပြီး လက် နှစ် ဘက် ဖြ <ငံ> ရေ နစ် နေ သူ ရဲ့

73: သူ ငယ် ချင်း လူ နာ ကြ <ည့်> ပေး လို့ ရ လား

79: H I V ရော ဂါ ပိုး စစ် ဖို့ ကို ကိုယ် တိုင် ခွ <ငံ> ပြု ကြောင်း လက် မှတ် ထိုး
ပါ ဗျ

92: အ စွယ် ရာ ကို တာ ဝန် ရှိ သူ တွေ က ကြ <ည့်> ကြ ပါ တယ် ခင် ဗျ

113: အ မျိုး က တော့ အ ချက် အ လက် တွေ ပေး ဖို့ စော <ငံ> ပေး ပါ ဦး ရှင်

117: ကြ <ည့်> ရ တာ ခင် ဗျား အ နီး မှုန် ဖြစ် နေ ပြီ

119: အ တွေ တွေ ရော ဂါ ကု သ ရေး ဌာ န ကို အ ရင် သွား ပြီး ကိုယ် ရေး အ ချက် ┘

၀ အ လက်

သွား ဖြ <ည့်> ပါ ရှင်

125: သ <ငံ> မှာ ရေ နဲ့ အ စား အ စာ လုံ လုံ လောက် လောက် ရှိ ရဲ့ လား

133: နောက် ကျော က နာ ကျင် ပြီး ခါး က နေ ဗိုက် အ ထိ ကို နာ ကျင် လိ <မ့်> မယ်

136: ကျွန် မ က အ ရေး ပေါ် ဌာ န ရဲ့ တာ <ဝန်> ခံ ဆ ရာ မ ဖြစ် ပါ တယ်

148: အ <ဝတ်> ချုပ် တဲ့ စက် ရုံ မှာ နေ ပါ တယ်

grep: write error: Broken pipe

အထက် output ကရလာတဲ့ line number တွေကို သုံးပြီး normalization မလုပ်ခင်ကဖိုင်မှာ ဝင်ရှာပြီး ရိုက်ထုတ်ခိုင်းကြည့်ရအောင်။ အဲဒါဆိုရင် ဘယ်လို ပြင်သွားတယ်ဆိုတာကို မြင်ရပါလိမ့်မယ်။

```
[16]: !grep -n "<" ./errors_train.txt | head -n 30 | awk -F: '{print $1"p"}' | sed -n 1,30p  
      ↪-f - train.my.normalize
```

grep: write error: Broken pipe

ပြီး တဲ့ အ ခါ နား ကြပ် ကို တံ တောင့် ဆစ် နား မှာ ထား မှာ မို့ ပါ ရှင်

ဒီ ကြား မှာ ခ ဣ လေး ထိုင် စောင့် ပေး လို့ ရ မ လား

တို့ တွေ အ နည်း ဆုံး သွား ကို ၃ - ၅ မိ နစ် အ ထိ အ နည်း ဆုံး တိုက် သင့် တယ် ရှင်
ဘယ် အ ချိန် က တည်း က သင် ၏ မျက် လုံး တွေ များ သည် ရဲ့ ရဲ့ နီ နေ ပြီး အ ရည်

↪တွေ နဲ့

ပြည့် နှက် နေ သ နည်း

သား အိမ် ခေါင်း ပွင့် တာ ကို သုံး သပ် ချင် လို့ ပါ

ငါ အ ပူ ချိန် မ တိုင်း ရ သေး ပေ မယ့် ကိုယ် ပူ နေ တယ် လို့ ခံ စား ရ တယ်

လူ နာ ရှင် ကို တဲ့ ခေါ် ထား ပါ တယ် ခင် ဗျာ လာ ပါ လိမ့် မယ်

ဘာ လုပ် သင့် တယ် ဆို တာ ပြော ဖို့ စော လွန်း နေ ပါ သေး တယ် N e u r o p s y c h

↪i

a t r y ဆို တာ က m e t a p h y s i c a l ဆေး တစ် မျိုး ပါ အ ခု အ ချိန် က စ

↪ပြီး

အ တူ တူ ကျော် ဖြတ် ကြ ရ အောင်

သင့် အ နေ နဲ့ ဖုန်း နဲ့ မ ဟုတ် ဘဲ အ ခြား ဘယ် လို နည်း လမ်း မျိုး နဲ့ ဆက် သွယ် စေ
ချင် ပါ သ လဲ

ပုံ မှန် အား ဖြင့် ကိုယ် တို့ ရဲ့ ကိုယ် အ ပူ ချိန် က ၃ ၇ . ၅ ဒီ ဂ ရီ ထက် မ ကျော်
ရ ဘူး

လက် ထိပ် မှာ အောက် ဆီ ဂျင် ပြည့် ၀ မှု သည် ၉ ၆ % ရှိ သည်

ရော ဂါ လက္ခဏာ က ရှောင် ဖြစ် နေ တာ နဲ့ တူ တယ်

ကိုယ် အား ဆန့် ပြီး ငြိမ် ငြိမ် နေ ပေး ပါ ရှင်

အ ပြင် မှာ စောင့် နေ ပေး လို့ ရ နိုင် မ လား

ဆ ရာ မ လေး ရှင်

ဓာတ် မ တည့် တာ က တစ် ဆင့် သ တိ လစ် တာ မျိုး ဖြစ် နိုင် ပါ တယ်

ဆ ရာ က လူ နာ ရဲ့ ဗိုက် ကို ဖိ နှိပ် ကြည့် ပါ မယ်

အ ဖျား ရှိ နေ တယ် နော် ရှင်

တော် တော် ကို ထိတ် လန့် ပြီး အော် ဟစ် အ ကူ အ ညီ ရှာ ပေ မယ့် ရပ် ကြည့် နေ ခဲ့ ကြ
တယ်

အောက် ကို ငုံ့ ပြီး လက် နှစ် ဘက် ဖြင့် ရေ နစ် နေ သူ ရဲ့

သူ ငယ် ချင်း လူ နာ ကြည့် ပေး လို့ ရ လား

H I V ရော ဂါ ပိုး စစ် ဖို့ ကို ကိုယ် တိုင် ခွင့် ပြု ကြောင်း လက် မှတ် ထိုး ပါ ဗျ

အ စွယ် ရာ ကို တာ ဝန် ရှိ သူ တွေ က ကြည့် ကြ ပါ တယ် ခင် ဗျ

အ မျိုး က တော့ အ ချက် အ လက် တွေ ပေး ဖို့ စောင့် ပေး ပါ ဦး ရှင်

ကြည့် ရ တာ ခင် ဗျား အ နီး မှန် ဖြစ် နေ ပြီ
အ ထွေ ထွေ ရော ဂါ ကု သ ရေး ဌာ န ကို အ ရင် သွား ပြီး ကိုယ် ရေး အ ချက် အ ပ
လက် သွား

ဖြည့် ပါ ရှင်
သင့် မှာ ရေ နဲ့ အ စား အ စာ လုံ လုံ လောက် လောက် ရှိ ရဲ့ လား
နောက် ကျော က နာ ကျင် ပြီး ခါး က နေ ဗိုက် အ ထိ ကို နာ ကျင် လိမ့် မယ်
ကျွန် မ က အ ရေး ပေါ် ဌာ န ရဲ့ တာ ဝန် ခံ ဆ ရာ မ ဖြစ် ပါ တယ်
အ ဝတ် ချုပ် တဲ့ စက် ရုံ မှာ နေ ပါ တယ်

အထက်မှာ မြင်နေရတဲ့ အမှားတွေထဲမှာ တကယ်က “ဝလုံး” နဲ့ “သုည” မှားရိုက်ထားတာမျိုးတွေလည်း ပါနေပါတယ်။ အဲဒီအပိုင်းတွေကိုလည်း `syl_normalizer.py` က ပြင်ပေးသွားပါလိမ့်မယ်။ ပြင်ဆင်သွားတဲ့ အသေးစိတ် အဆင့်တွေကို အောက်ပါ `log` ဖိုင်တွေကို ဝင်ကြည့်ရင် မြင်ရပါလိမ့်မယ်။

```
[17]: !ls corrections*.log
```

```
corrections_dev.log  corrections_test.log  corrections_train.log
```

```
[19]: !head -n 300 corrections_train.log
```

```
line      5 | 'တော င်း'          -> 'တောင့်'          |
stage3:merge_with_previous
line     10 | 'စော င်း'          -> 'စောင့်'          |
stage3:merge_with_previous
line     14 | 'သ င်း'            -> 'သင့်'            |
stage3:merge_with_previous
line     14 | 'ရှ င်း'            -> 'ရှင့်'            |
stage3:merge_with_previous
line     20 | 'ပြ ည့်'            -> 'ပြည့်'            |
stage3:merge_with_previous
line     21 | 'ပွ င်း'            -> 'ပွင့်'            |
stage3:merge_with_previous
line     24 | 'မ ယ့်'              -> 'မယ့်'              |
stage3:merge_with_previous
line     29 | 'လိ မ့်'             -> 'လိမ့်'             |
stage3:merge_with_previous
line     34 | 'သ င်း'            -> 'သင့်'            |
stage3:merge_with_previous
line     35 | 'သ င်း'            -> 'သင့်'            |
stage3:merge_with_previous
line     40 | 'ဖြ င်း'            -> 'ဖြင့်'            |
stage3:merge_with_previous
line     45 | 'ပြ ည့်'            -> 'ပြည့်'            |
stage3:merge_with_previous
line     47 | 'ဆ န်း'             -> 'ဆန့်'             |
stage3:merge_with_previous
```

line 52 'စော ငုံ'	-> 'စောငုံ'	
stage3:merge_with_previous		
line 55 'ရှ ငုံ'	-> 'ရှငုံ'	
stage3:merge_with_previous		
line 58 'တ ညှိ'	-> 'တညှိ'	
stage3:merge_with_previous		
line 58 'ဆ ငုံ'	-> 'ဆငုံ'	
stage3:merge_with_previous		
line 61 'ကြ ညှိ'	-> 'ကြညှိ'	
stage3:merge_with_previous		
line 65 'ရှ ငုံ'	-> 'ရှငုံ'	
stage3:merge_with_previous		
line 68 'လ နံ'	-> 'လနံ'	
stage3:merge_with_previous		
line 68 'မ ယုံ'	-> 'မယုံ'	
stage3:merge_with_previous		
line 68 'ကြ ညှိ'	-> 'ကြညှိ'	
stage3:merge_with_previous		
line 72 'ဖြ ငုံ'	-> 'ဖြငုံ'	
stage3:merge_with_previous		
line 73 'ကြ ညှိ'	-> 'ကြညှိ'	
stage3:merge_with_previous		
line 79 'ခွ ငုံ'	-> 'ခွငုံ'	
stage3:merge_with_previous		
line 92 'ကြ ညှိ'	-> 'ကြညှိ'	
stage3:merge_with_previous		
line 113 'စော ငုံ'	-> 'စောငုံ'	
stage3:merge_with_previous		
line 117 'ကြ ညှိ'	-> 'ကြညှိ'	
stage3:merge_with_previous		
line 119 'ဖြ ညှိ'	-> 'ဖြညှိ'	
stage3:merge_with_previous		
line 125 'သ ငုံ'	-> 'သငုံ'	
stage3:merge_with_previous		
line 133 'လိ မုံ'	-> 'လိမုံ'	
stage3:merge_with_previous		
line 156 'ရှ ငုံ'	-> 'ရှငုံ'	
stage3:merge_with_previous		
line 160 'နှ ငုံ'	-> 'နှငုံ'	
stage3:merge_with_previous		
line 166 'စော ငုံ'	-> 'စောငုံ'	
stage3:merge_with_previous		
line 174 'ရှ ငုံ'	-> 'ရှငုံ'	

stage3:merge_with_previous		
line 178 'ထ ည့်'	-> 'ထည့်'	
stage3:merge_with_previous		
line 179 'ကို ယ့်'	-> 'ကိုယ့်'	
stage3:merge_with_previous		
line 179 'ကျ င့်'	-> 'ကျင့်'	
stage3:merge_with_previous		
line 182 'ဆ င့်'	-> 'ဆင့်'	
stage3:merge_with_previous		
line 182 'ဆ င့်'	-> 'ဆင့်'	
stage3:merge_with_previous		
line 187 'မ ယ့်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 189 'သ န့်'	-> 'သန့်'	
stage3:merge_with_previous		
line 193 'မ ယ့်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 200 'တ ည့်'	-> 'တည့်'	
stage3:merge_with_previous		
line 200 'တ ည့်'	-> 'တည့်'	
stage3:merge_with_previous		
line 216 'လ န့်'	-> 'လန့်'	
stage3:merge_with_previous		
line 217 'စော င့်'	-> 'စောင့်'	
stage3:merge_with_previous		
line 217 'ကြ ည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 221 'သ င့်'	-> 'သင့်'	
stage3:merge_with_previous		
line 223 'ကြော င့်'	-> 'ကြောင့်'	
stage3:merge_with_previous		
line 224 'သ င့်'	-> 'သင့်'	
stage3:merge_with_previous		
line 224 'တ င့်'	-> 'တင့်'	
stage3:merge_with_previous		
line 225 'ကြ ည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 231 'မ ယ့်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 233 'တ န့်'	-> 'တန့်'	
stage3:merge_with_previous		
line 233 'ကြော င့်'	-> 'ကြောင့်'	
stage3:merge_with_previous		

line 248 ' ရှ င်'	-> ' ရှ င်'	
stage3:merge_with_previous		
line 251 ' ကျ င်'	-> ' ကျ င်'	
stage3:merge_with_previous		
line 258 ' ကြ ည့်'	-> ' ကြည့်'	
stage3:merge_with_previous		
line 258 ' ကြ ည့်'	-> ' ကြည့်'	
stage3:merge_with_previous		
line 258 ' ရှ င်'	-> ' ရှ င်'	
stage3:merge_with_previous		
line 263 ' မ ယ့်'	-> ' မယ့်'	
stage3:merge_with_previous		
line 266 ' စ ဉ်'	-> ' စဉ်'	
stage3:merge_with_previous		
line 266 ' ကြ ဉ်'	-> ' ကြဉ်'	
stage3:merge_with_previous		
line 268 ' မ ယ့်'	-> ' မယ့်'	
stage3:merge_with_previous		
line 282 ' ယှ ဉ်'	-> ' ယှဉ်'	
stage3:merge_with_previous		
line 282 ' ကြ ည့်'	-> ' ကြည့်'	
stage3:merge_with_previous		
line 283 ' ကြော င်'	-> ' ကြောင့်'	
stage3:merge_with_previous		
line 285 ' ကြော င်'	-> ' ကြောင့်'	
stage3:merge_with_previous		
line 288 ' မ ယ့်'	-> ' မယ့်'	
stage3:merge_with_previous		
line 293 ' မ ယ့်'	-> ' မယ့်'	
stage3:merge_with_previous		
line 297 ' ဆ င်'	-> ' ဆင့်'	
stage3:merge_with_previous		
line 300 ' တ ည့်'	-> ' တည့်'	
stage3:merge_with_previous		
line 308 ' ရှ င်'	-> ' ရှ င်'	
stage3:merge_with_previous		
line 316 ' စော င်'	-> ' စောင့်'	
stage3:merge_with_previous		
line 316 ' ကြ ည့်'	-> ' ကြည့်'	
stage3:merge_with_previous		
line 323 ' ဟ န့်'	-> ' ဟန့်'	
stage3:merge_with_previous		
line 325 ' ချ ဉ်'	-> ' ချဉ်'	

stage3:merge_with_previous		
line 328 'ရှ င်'	-> 'ရှ င်'	
stage3:merge_with_previous		
line 330 'ရှ င်'	-> 'ရှ င်'	
stage3:merge_with_previous		
line 336 'ရှ င်'	-> 'ရှ င်'	
stage3:merge_with_previous		
line 344 'စော င်'	-> 'စော င်'	
stage3:merge_with_previous		
line 346 'ကြော င်'	-> 'ကြော င်'	
stage3:merge_with_previous		
line 352 'စ ဉ်'	-> 'စ ဉ်'	
stage3:merge_with_previous		
line 357 'ရှ င်'	-> 'ရှ င်'	
stage3:merge_with_previous		
line 362 'ခွ င်'	-> 'ခွ င်'	
stage3:merge_with_previous		
line 364 'လိ မ်'	-> 'လိ မ်'	
stage3:merge_with_previous		
line 377 'ကြ ည်'	-> 'ကြ ည်'	
stage3:merge_with_previous		
line 377 'ကြ ည်'	-> 'ကြ ည်'	
stage3:merge_with_previous		
line 378 'ကြ ည်'	-> 'ကြ ည်'	
stage3:merge_with_previous		
line 380 'ချ ဉ်'	-> 'ချ ဉ်'	
stage3:merge_with_previous		
line 383 'မ ယ်'	-> 'မ ယ်'	
stage3:merge_with_previous		
line 388 'သ င်'	-> 'သ င်'	
stage3:merge_with_previous		
line 390 'သြော်'	-> 'သြော်'	
↪		
stage1:rule25 (သြော် -> သြော် (U+102A, AU independent vowel))		
line 406 'ရှ င်'	-> 'ရှ င်'	
stage3:merge_with_previous		
line 409 'စော င်'	-> 'စော င်'	
stage3:merge_with_previous		
line 412 'ထ ည်'	-> 'ထ ည်'	
stage3:merge_with_previous		
line 412 'လိ မ်'	-> 'လိ မ်'	
stage3:merge_with_previous		
line 420 'မ ယ်'	-> 'မ ယ်'	

stage3:merge_with_previous		
line 422 'သ န်'	-> 'သန်'	
stage3:merge_with_previous		
line 423 'သ င်'	-> 'သင့်'	
stage3:merge_with_previous		
line 423 'ရှ င်'	-> 'ရှင့်'	
stage3:merge_with_previous		
line 425 'ရှ င်'	-> 'ရှင့်'	
stage3:merge_with_previous		
line 431 'ကျ င်'	-> 'ကျင့်'	
stage3:merge_with_previous		
line 438 'မ ယ်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 449 'မ ယ်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 453 'ကြော င်'	-> 'ကြောင့်'	
stage3:merge_with_previous		
line 453 'မ ယ်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 453 'တော င်'	-> 'တောင့်'	
stage3:merge_with_previous		
line 466 'ကို ယ်'	-> 'ကိုယ့်'	
stage3:merge_with_previous		
line 475 'မ ယ်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 483 'သ င်'	-> 'သင့်'	
stage3:merge_with_previous		
line 485 'မ ယ်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 492 'သ င်'	-> 'သင့်'	
stage3:merge_with_previous		
line 494 'ပြ ည့်'	-> 'ပြည့်'	
stage3:merge_with_previous		
line 494 'အော င်'	-> 'အောင့်'	
stage3:merge_with_previous		
line 494 'အော င်'	-> 'အောင့်'	
stage3:merge_with_previous		
line 497 'ကြော င်'	-> 'ကြောင့်'	
stage3:merge_with_previous		
line 501 'ကို ယ်'	-> 'ကိုယ့်'	
stage3:merge_with_previous		
line 501 'ကို ယ်'	-> 'ကိုယ့်'	
stage3:merge_with_previous		
line 502 'ကြ ည့်'	-> 'ကြည့်'	

stage3:merge_with_previous		
line 503 ' ရှ င်'	-> ' ရှ င်'	
stage3:merge_with_previous		
line 515 ' လှ ည့်'	-> ' လှ ည့်'	
stage3:merge_with_previous		
line 520 ' မ ယ့်'	-> ' မယ့်'	
stage3:merge_with_previous		
line 523 ' ကြော င်'	-> ' ကြောငံ'	
stage3:merge_with_previous		
line 535 ' ကျ င်'	-> ' ကျငံ'	
stage3:merge_with_previous		
line 546 ' ကြော င်'	-> ' ကြောငံ'	
stage3:merge_with_previous		
line 551 ' သ င်'	-> ' သငံ'	
stage3:merge_with_previous		
line 555 ' ကြော င်'	-> ' ကြောငံ'	
stage3:merge_with_previous		
line 555 ' မ ယ့်'	-> ' မယ့်'	
stage3:merge_with_previous		
line 557 ' ရှ င်'	-> ' ရှ င်'	
stage3:merge_with_previous		
line 560 ' သ င်'	-> ' သငံ'	
stage3:merge_with_previous		
line 574 ' အော င်'	-> ' အောငံ'	
stage3:merge_with_previous		
line 579 ' စော င်'	-> ' စောငံ'	
stage3:merge_with_previous		
line 579 ' ကြ ည့်'	-> ' ကြည့်'	
stage3:merge_with_previous		
line 583 ' ထ ည့်'	-> ' ထည့်'	
stage3:merge_with_previous		
line 585 ' ကြော င်'	-> ' ကြောငံ'	
stage3:merge_with_previous		
line 590 ' လိ မ့်'	-> ' လိမ့်'	
stage3:merge_with_previous		
line 591 ' ကို ယ့်'	-> ' ကိုယ့်'	
stage3:merge_with_previous		
line 592 ' စော င်'	-> ' စောငံ'	
stage3:merge_with_previous		
line 595 ' ရှ င်'	-> ' ရှ င်'	
stage3:merge_with_previous		
line 600 ' ကို ယ့်'	-> ' ကိုယ့်'	
stage3:merge_with_previous		

line 600 'တော ငုံ'	-> 'တောငုံ'	
stage3:merge_with_previous		
line 601 'ရှ ငုံ'	-> 'ရှငုံ'	
stage3:merge_with_previous		
line 612 'တ ညှိ'	-> 'တညှိ'	
stage3:merge_with_previous		
line 619 'လိ မုံ'	-> 'လိမုံ'	
stage3:merge_with_previous		
line 624 'ကြော ငုံ'	-> 'ကြောငုံ'	
stage3:merge_with_previous		
line 636 'ပြ ညှိ'	-> 'ပြညှိ'	
stage3:merge_with_previous		
line 653 'စိ မုံ'	-> 'စိမုံ'	
stage3:merge_with_previous		
line 653 'စိ မုံ'	-> 'စိမုံ'	
stage3:merge_with_previous		
line 657 'လိ မုံ'	-> 'လိမုံ'	
stage3:merge_with_previous		
line 659 'ရှ ငုံ'	-> 'ရှငုံ'	
stage3:merge_with_previous		
line 666 'ဆ ငုံ'	-> 'ဆငုံ'	
stage3:merge_with_previous		
line 671 'သ ငုံ'	-> 'သငုံ'	
stage3:merge_with_previous		
line 678 'ကြ ညှိ'	-> 'ကြညှိ'	
stage3:merge_with_previous		
line 681 'သ နံ'	-> 'သနံ'	
stage3:merge_with_previous		
line 681 'သ နံ'	-> 'သနံ'	
stage3:merge_with_previous		
line 683 'စော ငုံ'	-> 'စောငုံ'	
stage3:merge_with_previous		
line 696 'စော ငုံ'	-> 'စောငုံ'	
stage3:merge_with_previous		
line 697 'ရှ ငုံ'	-> 'ရှငုံ'	
stage3:merge_with_previous		
line 703 'သ နံ'	-> 'သနံ'	
stage3:merge_with_previous		
line 703 'စ ညှိ'	-> 'စညှိ'	
stage3:merge_with_previous		
line 703 'သ ငုံ'	-> 'သငုံ'	
stage3:merge_with_previous		
line 703 'စ ညှိ'	-> 'စညှိ'	
stage3:merge_with_previous		

line 704 'ကြောင့်'	-> 'ကြောင့်'	
stage3:merge_with_previous		
line 722 'ရှင့်'	-> 'ရှင့်'	
stage3:merge_with_previous		
line 730 'ဖြည့်'	-> 'ဖြည့်'	
stage3:merge_with_previous		
line 731 'ရှင့်'	-> 'ရှင့်'	
stage3:merge_with_previous		
line 740 'သင့်'	-> 'သင့်'	
stage3:merge_with_previous		
line 746 'အောင့်'	-> 'အောင့်'	
stage3:merge_with_previous		
line 751 'ကြည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 756 'ဟင့်'	-> 'ဟင့်'	
stage3:merge_with_previous		
line 760 'သင့်'	-> 'သင့်'	
stage3:merge_with_previous		
line 771 'သင့်'	-> 'သင့်'	
stage3:merge_with_previous		
line 772 'သင့်'	-> 'သင့်'	
stage3:merge_with_previous		
line 777 'မည့်'	-> 'မည့်'	
stage3:merge_with_previous		
line 784 'ကြောင့်'	-> 'ကြောင့်'	
stage3:merge_with_previous		
line 793 'ကြည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 796 'ကြည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 801 'စောင့်'	-> 'စောင့်'	
stage3:merge_with_previous		
line 811 'မြင့်'	-> 'မြင့်'	
stage3:merge_with_previous		
line 811 'မယ့်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 811 'သင့်'	-> 'သင့်'	
stage3:merge_with_previous		
line 817 'ကျင့်'	-> 'ကျင့်'	
stage3:merge_with_previous		
line 822 'နှင့်'	-> 'နှင့်'	
stage3:merge_with_previous		
line 824 'အောင့်'	-> 'အောင့်'	
stage3:merge_with_previous		

line 826 'လိ မ့်'	-> 'လိမ့်'	
stage3:merge_with_previous		
line 826 'ရ င့်'	-> 'ရင့်'	
stage3:merge_with_previous		
line 826 'လိ မ့်'	-> 'လိမ့်'	
stage3:merge_with_previous		
line 830 'သ န်'	-> 'သန်'	
stage3:merge_with_previous		
line 831 'ကြ ည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 837 'ကို ယ့်'	-> 'ကိုယ့်'	
stage3:merge_with_previous		
line 842 'ကြ ည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 843 'ထ ည့်'	-> 'ထည့်'	
stage3:merge_with_previous		
line 845 'ကြ ည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 847 'ရှ င့်'	-> 'ရှင့်'	
stage3:merge_with_previous		
line 866 'ကြ ည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 870 'သ န်'	-> 'သန်'	
stage3:merge_with_previous		
line 883 'သ န်'	-> 'သန်'	
stage3:merge_with_previous		
line 883 'နှ င့်'	-> 'နှင့်'	
stage3:merge_with_previous		
line 886 'ဖြ င့်'	-> 'ဖြင့်'	
stage3:merge_with_previous		
line 891 'ကြ ည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 902 'လ န်'	-> 'လန်'	
stage3:merge_with_previous		
line 906 'မ ယ့်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 924 'ရှ င့်'	-> 'ရှင့်'	
stage3:merge_with_previous		
line 933 'ဆ န်'	-> 'ဆန်'	
stage3:merge_with_previous		
line 939 'နှ င့်'	-> 'နှင့်'	
stage3:merge_with_previous		
line 939 'သ ည့်'	-> 'သည့်'	

stage3:merge_with_previous		
line 940 'မ ည့်'	-> 'မည့်'	
stage3:merge_with_previous		
line 940 'န င်'	-> 'နင့်'	
stage3:merge_with_previous		
line 960 'ရှ င်'	-> 'ရှင့်'	
stage3:merge_with_previous		
line 961 'လိ မ့်'	-> 'လိမ့်'	
stage3:merge_with_previous		
line 968 'ကြ ည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 968 'ဖ င်'	-> 'ဖင့်'	
stage3:merge_with_previous		
line 971 'ကြ ည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 971 'ကြ ည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 971 'ကို ယ့်'	-> 'ကိုယ့်'	
stage3:merge_with_previous		
line 981 'က ယ့်'	-> 'ကယ့်'	
stage3:merge_with_previous		
line 988 'ကြော င်'	-> 'ကြောင့်'	
stage3:merge_with_previous		
line 1005 'ကြော င်'	-> 'ကြောင့်'	
stage3:merge_with_previous		
line 1006 'ကြ ည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 1007 'မ ယ့်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 1013 'ဆော င်'	-> 'ဆောင့်'	
stage3:merge_with_previous		
line 1015 'သ ည့်'	-> 'သည့်'	
stage3:merge_with_previous		
line 1026 'မ ယ့်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 1028 'ဆ င်'	-> 'ဆင့်'	
stage3:merge_with_previous		
line 1028 'ဆ င်'	-> 'ဆင့်'	
stage3:merge_with_previous		
line 1036 'ရှ င်'	-> 'ရှင့်'	
stage3:merge_with_previous		
line 1047 'ဆ င်'	-> 'ဆင့်'	
stage3:merge_with_previous		

line 1058 'လိ မှ်'	-> 'လိမှ်'	
stage3:merge_with_previous		
line 1059 'တ ညှ်'	-> 'တညှ်'	
stage3:merge_with_previous		
line 1061 'ရ င်'	-> 'ရင့်'	
stage3:merge_with_previous		
line 1070 'သ င်'	-> 'သင့်'	
stage3:merge_with_previous		
line 1070 'သ ညှ်'	-> 'သညှ်'	
stage3:merge_with_previous		
line 1076 'ကြ ညှ်'	-> 'ကြညှ်'	
stage3:merge_with_previous		
line 1076 'ရှ င်'	-> 'ရှင့်'	
stage3:merge_with_previous		
line 1083 'စော င်'	-> 'စောင့်'	
stage3:merge_with_previous		
line 1084 'ကြော င်'	-> 'ကြောင့်'	
stage3:merge_with_previous		
line 1092 'ဆ င်'	-> 'ဆင့်'	
stage3:merge_with_previous		
line 1104 'စော င်'	-> 'စောင့်'	
stage3:merge_with_previous		
line 1106 'ကျ င်'	-> 'ကျင့်'	
stage3:merge_with_previous		
line 1106 'သ ညှ်'	-> 'သညှ်'	
stage3:merge_with_previous		
line 1110 'သ င်'	-> 'သင့်'	
stage3:merge_with_previous		
line 1112 'ဖ န်'	-> 'ဖန်'	
stage3:merge_with_previous		
line 1112 'ဖ န်'	-> 'ဖန်'	
stage3:merge_with_previous		
line 1119 'စ ဉ်'	-> 'စဉ်'	
stage3:merge_with_previous		
line 1125 'စော င်'	-> 'စောင့်'	
stage3:merge_with_previous		
line 1127 'မ ယ်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 1128 'ကြော င်'	-> 'ကြောင့်'	
stage3:merge_with_previous		
line 1131 'အော င်'	-> 'အောင့်'	
stage3:merge_with_previous		
line 1134 'ဪ'	-> 'ဪ'	

↪ |

▮

```

stage1:rule25 ( ဪ -> ဪ (U+102A, AU independent vowel))
line 1138 | ' ကြောင့် ' -> ' ကြောင့် ' |
stage3:merge_with_previous
line 1141 | ' ကြောင့် ' -> ' ကြောင့် ' |
stage3:merge_with_previous
line 1141 | ' မ ယ့် ' -> ' မယ့် ' |
stage3:merge_with_previous
line 1142 | ' သ န့် ' -> ' သန့် ' |
stage3:merge_with_previous
line 1148 | ' ကြောင့် ' -> ' ကြောင့် ' |
stage3:merge_with_previous
line 1162 | ' ကို ယ့် ' -> ' ကိုယ့် ' |
stage3:merge_with_previous
line 1162 | ' ပြ ည့် ' -> ' ပြည့် ' |
stage3:merge_with_previous
line 1172 | ' သ င့် ' -> ' သင့် ' |
stage3:merge_with_previous
line 1189 | ' ကြ ည့် ' -> ' ကြည့် ' |
stage3:merge_with_previous
line 1194 | ' ကြောင့် ' -> ' ကြောင့် ' |
stage3:merge_with_previous
line 1194 | ' ကြောင့် ' -> ' ကြောင့် ' |
stage3:merge_with_previous
line 1197 | ' သ န့် ' -> ' သန့် ' |
stage3:merge_with_previous
line 1204 | ' ကြ ည့် ' -> ' ကြည့် ' |
stage3:merge_with_previous
line 1205 | ' မ ည့် ' -> ' မည့် ' |
stage3:merge_with_previous
line 1206 | ' သ င့် ' -> ' သင့် ' |
stage3:merge_with_previous
line 1209 | ' စော င့် ' -> ' စောင့် ' |
stage3:merge_with_previous
line 1211 | ' ကြ ည့် ' -> ' ကြည့် ' |
stage3:merge_with_previous
line 1220 | ' ရှ င့် ' -> ' ရှင့် ' |
stage3:merge_with_previous
line 1233 | ' လိ မ့် ' -> ' လိမ့် ' |
stage3:merge_with_previous
line 1234 | ' ဖြ င့် ' -> ' ဖြင့် ' |
stage3:merge_with_previous
line 1234 | ' ကြ ည့် ' -> ' ကြည့် ' |
stage3:merge_with_previous

```

line 1237 'တ ည့်'	-> 'တည့်'	
stage3:merge_with_previous		
line 1247 'ကြော င်'	-> 'ကြောင့်'	
stage3:merge_with_previous		
line 1249 'ကြ ည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 1249 'ရှ င်'	-> 'ရှင့်'	
stage3:merge_with_previous		
line 1250 'သ င်'	-> 'သင့်'	
stage3:merge_with_previous		
line 1266 'ဆ န်'	-> 'ဆန့်'	
stage3:merge_with_previous		
line 1267 'ဆ င်'	-> 'ဆင့်'	
stage3:merge_with_previous		
line 1285 'မ ယ့်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 1286 'အော င်'	-> 'အောင့်'	
stage3:merge_with_previous		
line 1287 'ကြ ည့်'	-> 'ကြည့်'	
stage3:merge_with_previous		
line 1291 'မ ယ့်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 1300 'ကြော င်'	-> 'ကြောင့်'	
stage3:merge_with_previous		
line 1308 'မ ယ့်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 1309 'ပြ ည့်'	-> 'ပြည့်'	
stage3:merge_with_previous		
line 1315 'ရ င်'	-> 'ရင့်'	
stage3:merge_with_previous		
line 1329 'ရှ င်'	-> 'ရှင့်'	
stage3:merge_with_previous		
line 1338 'ခွ င်'	-> 'ခွင့်'	
stage3:merge_with_previous		
line 1341 'စော င်'	-> 'စောင့်'	
stage3:merge_with_previous		
line 1346 'ဖြ င်'	-> 'ဖြင့်'	
stage3:merge_with_previous		
line 1349 'ရှ င်'	-> 'ရှင့်'	
stage3:merge_with_previous		
line 1355 'မ ယ့်'	-> 'မယ့်'	
stage3:merge_with_previous		
line 1355 'ခွ င်'	-> 'ခွင့်'	

```

stage3:merge_with_previous
line 1359 | 'ကြည့်' -> 'ကြည့်' |
stage3:merge_with_previous
line 1378 | 'လိမ့်' -> 'လိမ့်' |
stage3:merge_with_previous
line 1379 | 'ဖြင့်' -> 'ဖြင့်' |
stage3:merge_with_previous
line 1379 | 'ဖြင့်' -> 'ဖြင့်' |
stage3:merge_with_previous
line 1390 | 'ရှင့်' -> 'ရှင့်' |
stage3:merge_with_previous
line 1403 | 'ကိုယ့်' -> 'ကိုယ့်' |
stage3:merge_with_previous

```

လက်ရှိ မြန်မာ-ထိုင်း corpus ရဲ့ အမှားအများစုကတော့ အသတ်နဲ့ အောက်ကမြစ် (အောက်မြစ်) ရိုက်တဲ့အစီအစဉ် လွဲနေတာ အများစုလို့ သိရှိရပါတယ်။

1.6 Filename Changing

Normalization လုပ်ထားတဲ့ ဖိုင်တွေကို Moses နဲ့ Run ဖို့အတွက် ဖိုင်နာမည်ပြောင်းသိမ်းတာ လုပ်ပါမယ်။

```

[20]: %pwd
[20]: '/home/ye/exp/SMT-NMT_tutorial/clean-data/big/normalize'

[22]: %mkdir data

[23]: !cp *.normalize ./data/

[24]: %cd data
/home/ye/exp/SMT-NMT_tutorial/clean-data/big/normalize/data

[25]: !wc *
10000 115810 1124813 dev.my.normalize
8000 92231 894623 test.my.normalize
62625 721810 7016052 train.my.normalize
80625 929851 9035488 total

[26]: !mv train.my.normalize train.my

[27]: !mv dev.my.normalize dev.my

[ ]: !mv test.my.normalize test.my

```

ထိုင်း ဒေတာဖိုင်တွေလည်း ကော်ပီကူးရလိမ့်မယ်။

```
[29]: !cp ../*.th .
```

```
[30]: !wc *.{my,th}
```

```
10000 115810 1124813 dev.my
8000 92231 894623 test.my
62625 721810 7016052 train.my
10000 71013 826135 dev.th
8000 56113 652038 test.th
62625 439436 5111625 train.th
161250 1496413 15625286 total
```

1.7 Prepare test-sgm Folder

ပုံမှန်အတိုင်း commandline တစ်ခုချင်းစီပေးရင်း PBSMT လုပ်ရင် မလိုပေမဲ့ moses version 2 ရဲ့ experiment management system ကို သုံးမယ်ဆိုရင် နောက်ထပ် အရေးကြီးတဲ့ အဆင့်တစ်ခုက SGM format အဖြစ် test ဖိုင်တွေကို ပြောင်းပေးရတဲ့ အဆင့်ပါပဲ။ အဲဒါကို normalize လုပ်ထားတဲ့ ဒေတာတွေအတွက် ပြင်ကြရဦးမယ်။

```
[45]: %cd /home/ye/exp/SMT-NMT_tutorial/clean-data/big/normalize/data
```

```
/home/ye/exp/SMT-NMT_tutorial/clean-data/big/normalize/data
```

```
[46]: !ls
```

```
dev.my dev.th test.my test.th train.my train.th
```

```
[47]: %mkdir test-sgm
```

ဆရာ စက်ထဲမှာက SGM ဖောမတ်ပြောင်းပေးမယ့် perl script တွေက ရှိပြီးသားမို့ အဆင်ပြေတဲ့ path ကနေပဲ အောက်ပါအတိုင်း ကော်ပီကူးယူလိုက်ပါတယ်။

```
[51]: !cp /home/ye/exp/SMT-NMT_tutorial/clean-data/big/test-sgm/*.pl ./test-sgm/
```

```
[52]: !ls ./test-sgm/*
```

```
./test-sgm/generate_sgms.pl ./test-sgm/ref2sgm.pl ./test-sgm/src2sgm.pl
```

```
[53]: %cd ./test-sgm/
```

```
/home/ye/exp/SMT-NMT_tutorial/clean-data/big/normalize/data/test-sgm
```

```
[54]: !cat ./generate_sgms.pl
```

```
#!/usr/bin/perl
use strict;

# written by Ye, NECTEC
# for MTRSS, YTU, Myanmar

my @langs;
```

```

foreach my $trainFile ( </home/ros/experiment/my-rk/data/train.[a-z][a-z]> )
foreach my $trainFile ( <../train.[a-z][a-z]> )
{
    $trainFile =~ m/train.([a-z][a-z])/;
    push @langs, $1;
}

foreach my $lang (@langs)
{
    `perl ./ref2sgm.pl $lang > test.$lang.ref.sgm`;
    `perl ./src2sgm.pl $lang > test.$lang.src.sgm`;
}

```

[55]: !cat ref2sgm.pl

```

#!/usr/bin/perl
use strict;

# written by Ye, NECTEC
# for MTRSS, YTU, Myanmar

my $trg = shift;

#print "<refset trglang=\"$trg\" setid=\"sl_data\" srclang=\"any\">\n";
print "<refset trglang=\"$trg\" setid=\"Thai-Myanmar_data\" srclang=\"any\">\n";
print "<doc sysid=\"ref\" docid=\"none\" genre=\"8000\" origlang=\"any\">\n";

#open FILE, "/home/ros/experiment/my-rk/data/test.$trg" or die;
open FILE, "../test.$trg" or die;

my $id=1;

while( <FILE> )
{
    chomp;

    print "<seg id=\"$id\">$_ </seg>\n";
    $id++;
}

print "</doc>\n</refset>\n";

```

[56]: !cat ./src2sgm.pl

```

#!/usr/bin/perl
use strict;

# written by Ye, NECTEC
# for MTRSS, YTU, Myanmar

```

```

my $src = shift;

print "<srcset setid=\"Thai-Myanmar_data\" srclang=\"any\">\n";
print "<doc docid=\"none\" genre=\"8000\" origlang=\"$src\">\n";

#open FILE, "/home/ros/experiment/my-rk/data/test.$src" or die;
open FILE, "../test.$src" or die;

my $id=1;

while( <FILE> )
{
    chomp;

    print "<seg id=\"$id\">$_ </seg>\n";
    $id++;
}

print "</doc>\n</srcset>\n";

```

အထက်မှာ မြင်ရတဲ့အတိုင်း ဒီ perl ဖိုင်တွေကတော့ ဝင်ပြင်စရာ မလိုပါဘူး။
SGM format အဖြစ်ပြောင်းထားတဲ့ test ဖိုင်တွေ ရအောင် run မယ်။

[57]:

```
!ls

generate_sgms.pl  ref2sgm.pl  src2sgm.pl
```

[60]:

```
!perl ./generate_sgms.pl

Run တာက error မတက်ရင် အောက်ပါအတိုင်း SGM ဖိုင်တွေထွက်လာပါလိမ့်မယ်။
```

[59]:

```
!ls

generate_sgms.pl  src2sgm.pl          test.my.src.sgm  test.th.src.sgm
ref2sgm.pl        test.my.ref.sgm  test.th.ref.sgm

output SGM ဖိုင်တွေကို စစ်ဆေးကြည့်ပါ။
```

[61]:

```
!head -n 20 test.my.ref.sgm

<refset trglang="my" setid="Thai-Myanmar_data" srclang="any">
<doc sysid="ref" docid="none" genre="8000" origlang="any">
<seg id="1">ခွဲ မှ ရ မှာ လား </seg>
<seg id="2">မျက် ခံ လှုပ် တဲ့ လက္ခဏာ ရှိ မယ် </seg>
<seg id="3">အ ရက် ကို စွန့် လွှတ် ရ မည် ဖြစ် ပြီး ပ ရှိ တင်း နှင့် ဗီ တာ မင်
ဓာတ် ကြွယ် ဝ သော အာ ဟာ ရ ရှိ သော အ စား အ စာ များ ကို စား သင့် သည် </seg>
<seg id="4">ဟုတ် ကဲ့ သူ ဖျား ပါ တယ် သူ သီး သန့် တော့ မ နေ ခဲ့ ဘူး </seg>
<seg id="5">မောင် လေး နောက် ဆုံး အ ကြိမ် ဆေး သောက် ခဲ့ တာ ဘယ် ချိန် လဲ </seg>
```

```

<seg id="6">ကိုယ် မှာ ကူး စက် တာ က နေ ဦး နောက် ထိ ပါ ကူး စက် လာ တယ် </
<seg>
<seg id="7">မစ် စ တာ ပ ရာ မို့ ရဲ့ ဆွေ မျိုး ဝင် လာ ဖို့ တစ် ချက် ခေါ် ပေး ပါ
ရှင် </seg>
<seg id="8">အ ချို့ ကို လျော့ ချ ကြည့် လိုက် ပါ </seg>
<seg id="9">သူ တို့ နဲ့ အ တူ တူ ရှိ နေ တယ် လို့ ပဲ တွေး မိ နေ တယ် ဗျာ </seg>
<seg id="10">ဆေး ယူ ပြီး ပြီ ဆို ရင် တော့ အိမ် ပြန် လို့ ရ ပြီ နော် </seg>
<seg id="11">ကျွန် မ သွေး အား နည်း သော ရော ဂါ ရှိ တယ် </seg>
<seg id="12">လွန် ခဲ့ တဲ့ နှစ် က ကိုယ့် ဌာ န မန် နေ ဂျာ က ကိုယ် အ မှား လုပ် ခဲ့
တာ တွေ အ တွက် အ ပြစ် စ တင် ထဲ က ပါ </seg>
<seg id="13">ဒီ အ ချိန် ထဲ လိင် ဆက် ဆံ မယ် ဆို ရင် လည်း ကွန် ခုံး သေ ချာ သုံး ပါ
</seg>
<seg id="14">ခွဲ ထား တာ ကို ဂ ရ စိုက် လို့ ရ အောင် ဆ ရာ ဝန် က ကပ် ပေး လိမ့်
<seg>
</seg>
<seg id="15">မှတ် ပုံ တင် ပြ လို့ ရ မ လား ရှင့် </seg>
<seg id="16">အ ထီး ကျန် ရင် ရော ကြောက် တတ် လား </seg>
<seg id="17">V D R L နဲ့ T P H A စစ် ဆေး ချက် က ပေါ့ စ တစ် ထွက် ပါ တယ် </seg>
<seg id="18">မာ ကျူ ရီ က ဓါ တု တစ် မျိုး ပါ </seg>

```

[62] : `!head -n 20 ./test.th.src.sgm`

```

<srcset setid="Thai-Myanmar_data" srclang="any">
<doc docid="none" genre="8000" origlang="th">
<seg id="1"> </seg>
<seg id="2"> </seg>
<seg id="3">
</seg>
<seg id="4"> </seg>
<seg id="5"> </seg>
<seg id="6"> </seg>
<seg id="7"> </seg>
<seg id="8"> </seg>
<seg id="9"> </seg>
<seg id="10"> </seg>
<seg id="11"> </seg>
<seg id="12">
</seg>
<seg id="13"> </seg>
<seg id="14"> </seg>
<seg id="15"> </seg>
<seg id="16"> </seg>
<seg id="17"> VDRL TPHA </seg>
<seg id="18"> </seg>

```

မြန်မာစာ ဒေတာတွေကို Normalization လုပ်တာ ပြီးသွားပြီ။ ဒီ Tutorial မှာတော့ မြန်မာစာဘက်အခြမ်းကိုပဲ

Normalization လုပ်ပါမယ်။ အချိန်လည်း သိပ်မရှိတာကြောင့် ထိုင်းဘက်အခြမ်းကတော့ ရှေ့က SMT-Tutorial-for-AI-Class-2.ipynb တုန်းက လုပ်ခဲ့တဲ့အတိုင်းပဲ ယူသုံးပါမယ်။

လက်ရှိ path ကို မှတ်ထားပြီး အဲဒီ path ကို configuration ထုတ်ဖို့အတွက် လိုအပ်တဲ့ perl script တွေမှာ ဝင်ပြင်ကြရပါလိမ့်မယ်။

```
[31]: %pwd
```

```
[31]: '/home/ye/exp/SMT-NMT_tutorial/clean-data/big/normalize/data'
```

အရင်ဆုံး experiment လုပ်မယ့် ဖိုလ်ဒါကို ပြင်ဆင်ကြရအောင်။

```
[32]: %cd /home/ye/exp/SMT-NMT_tutorial
```

```
/home/ye/exp/SMT-NMT_tutorial
```

```
[33]: %mkdir pbsmt-big-normalize
```

```
[34]: %cd pbsmt-big-normalize
```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt-big-normalize
```

Moses ကို run ဖို့ configuration ဖိုင်တွေထုတ်ပေးမယ့် perl script တချို့ကို အရင်တခေါက် run ခဲ့တဲ့ pbsmt-big ဖိုလ်ဒါအောက်ကနေ လက်ရှိ path အောက်ကို ကော်ပီကူးယူပါမယ်။

```
[35]: %cp ../pbsmt-big/*.pl .
```

```
[36]: !ls
```

```
generate_configs.pl  run-baseline.pl
```

```
[37]: %cp ../pbsmt-big/config.baseline .
```

```
[38]: !ls
```

```
config.baseline  generate_configs.pl  run-baseline.pl
```

အထက်မှာ မြင်ရတဲ့အတိုင်း config.baseline, generate_configs.pl နဲ့ run-baseline.pl ဖိုင်သုံးဖိုင်စလုံး လက်ရှိ path အောက်ကို ကော်ပီကူးပြီးသွားပြီ။

ဆရာစက်ထဲမှာက တခြား လိုအပ်တဲ့ setting တွေအတွက်က ရှေ့က နှစ်ခေါက် (အတန်းထဲမှာ တစ်ခေါက်၊ SMT-Tutorial-for-AI-Class-2.ipynb တုန်းက တစ်ခေါက်) run ထားပြီးပြီမို့ တကယ်တမ်း ဝင်ပြင်ရမှာက အောက်ပါ path နှစ်ခုပဲ ရှိပါလိမ့်မယ်။

Experiment Path: /home/ye/exp/SMT-NMT_tutorial/pbsmt-big-normalize

Parallel Corpus Path: /home/ye/exp/SMT-NMT_tutorial/clean-data/big/normalize/data

ဆရာ terminal မှာပဲ ဖိုင်တွေကို ဖွင့်ပြီး ဝင်ပြင်ခဲ့တယ်။

ပြင်ထားပြီးသာ perl, config ဖိုင်တွေက အောက်ပါအတိုင်းပါ။

```
[39]: !cat ./config.baseline
```

```

### directories that contain tools and data
#
# moses
#moses-src-dir = /home/ros/mosesdecoder
#moses-src-dir = /home/lar/tool/moses/
#moses-src-dir = /home/ye/tool/mosesbin/ubuntu-17.04/moses
moses-src-dir = /home/ye/tool/mosesbin/ubuntu-17.04/moses

# moses binaries
moses-bin-dir = $moses-src-dir/bin

# moses scripts
moses-script-dir = $moses-src-dir/scripts

# directory where GIZA++/MGIZA programs resides
#external-bin-dir = /home/lar/tool/giza-pp-master/mkcls-v2
#external-bin-dir = /home/lar/tool/giza-pp-master/GIZA++-v2
#external-bin-dir = /home/lar/tool/giza-pp-master/
#external-bin-dir = /home/ye/tool/mgiza/mgizapp/bin
external-bin-dir = /home/ye/tool/giza-pp/GIZA++-v2

# srilm
#srilm-dir = /data/lttools/training/srilm/srilm-1.6.0/bin/i686-m64
#I haven't installed SRILM yet on Deep Learning Box computer
#srilm-dir = /home/ros/tool/srilm-1.7.1/bin/i686-m64

# firstlm
#firstlm-dir = $moses-src-dir/firstlm/bin

# randlm
#randlm-dir = $moses-src-dir/randlm/bin

# kenlm
lmplz = $moses-bin-dir/lmplz

# data
#myrk-data = /home/lar/experiment/my-rk/smt1/t9
#mykc-data = /home/lar/experiment/kachin-myanmar/demo-mykc-smt/4demo/
#mykc-data = /home/lar/experiment/kachin-myanmar/demo-mykc-smt/4demo2/
#myrk-data = /media/lar/Transcend/student/lecture/mtrss/pbsmt-demo/pbsmt/data/
thmy-data = /home/ye/exp/SMT-NMT_tutorial/clean-data/big/normalize/data

### basic tools
#
# moses decoder
decoder = $moses-bin-dir/moses

```

```

# conversion of phrase table into binary on-disk format
#ttable-binarizer = $moses-bin-dir/processPhraseTable
#ttable-binarizer = $moses-bin-dir/processPhraseTableMin

# conversion of rule table into binary on-disk format
#ttable-binarizer = "$moses-bin-dir/CreateOnDiskPt 1 1 4 100 2"

# tokenizers - comment out if all your data is already tokenized
#input-tokenizer = "$moses-script-dir/tokenizer/tokenizer.perl -a -l $input-
extension"
#output-tokenizer = "$moses-script-dir/tokenizer/tokenizer.perl -a -l $output-
extension"

# truecasers - comment out if you do not use the truecaser
#input-truecaser = $moses-script-dir/recaser/truecase.perl
#output-truecaser = $moses-script-dir/recaser/truecase.perl
#dettruecaser = $moses-script-dir/recaser/dettruecase.perl

### generic parallelizer for cluster and multi-core machines
# you may specify a script that allows the parallel execution
# parallizable steps (see meta file). you also need specify
# the number of jobs (cluster) or cores (multicore)
#
generic-parallelizer = $moses-script-dir/ems/support/generic-parallelizer.perl
#generic-parallelizer = $moses-script-dir/ems/support/generic-multicore-
parallelizer.perl

### cluster settings (if run on a cluster machine)
# number of jobs to be submitted in parallel
#
#jobs = 10

# arguments to qsub when scheduling a job
#qsub-settings = ""

# project for privileges and usage accounting
#qsub-project = iccs_smt

# memory and time
#qsub-memory = 4
#qsub-hours = 48

### multi-core settings
# when the generic parallelizer is used, the number of cores
# specified here
#cores = 8

```

```
#####
```

```

# PARALLEL CORPUS PREPARATION:
# create a tokenized, sentence-aligned corpus, ready for training

[CORPUS]

### long sentences are filtered out, since they slow down GIZA++
# and are a less reliable source of data. set here the maximum
# length of a sentence
#
max-sentence-length = 80

[CORPUS:thmy]

### command to run to get raw corpus files
#
# get-corpus-script =

### raw corpus files (untokenized, but sentence aligned)
#
raw-stem = $thmy-data/train

### tokenized corpus files (may contain long sentences)
#
#tokenized-stem = $myrk-data/train

### if sentence filtering should be skipped,
# point to the clean training data
#
#clean-stem =

### if corpus preparation should be skipped,
# point to the prepared training data
#
#lowercased-stem =

#####
# LANGUAGE MODEL TRAINING

[LM]

### tool to be used for language model training
# srilm
#lm-training = /data/lttools/training/srilm/srilm-1.6.0/bin/i686-m64/ngram-count
#lm-training = /home/ros/tool/srilm-1.7.1/bin/i686-m64/ngram-count
#settings = "-interpolate -kndiscount -unk"

# firstlm training
# msb = modified kneser ney; p=0 no singleton pruning

```

```

#lm-training = "$moses-script-dir/generic/trainlm-irst2.perl -cores $cores
-irst-dir $irstlm-dir -temp-dir $working-dir/tmp"
#settings = "-s msb -p 0"

# kenlm
# additional parameters to lmplz (check lmplz help message)
#settings = "-T $working-dir/tmp -S 10G"
# this tells EMS to use lmplz and tells EMS where lmplz is located
#lm-training = "$moses-script-dir/generic/trainlm-lmplz.perl -lmplz $lmplz"
#lm-training = "$moses-script-dir/ems/support/lmplz-wrapper.perl -bin $moses-
bin-dir/lmplz"

# kenlm training
lm-training = "$moses-script-dir/ems/support/lmplz-wrapper.perl -bin $moses-bin-
dir/lmplz"
settings = "--prune '0 0 1' -T $working-dir/lm -S 20% --discount_fallback"

#lm-binarizer = $moses-bin-dir/build_binary

# order of the language model
order = 5

### tool to be used for training randomized language model from scratch
# (more commonly, a SRILM is trained)
#
#rlm-training = "$randlm-dir/buildlm -falsepos 8 -values 8"

### script to use for binary table format for irstlm or kenlm
# (default: no binarization)

# irstlm
#lm-binarizer = $irstlm-dir/compile-lm

# kenlm, also set type to 8
lm-binarizer = $moses-bin-dir/build_binary
type = 8

### script to create quantized language model format (irstlm)
# (default: no quantization)
#
#lm-quantizer = $irstlm-dir/quantize-lm

### script to use for converting into randomized table format
# (default: no randomization)
#
#lm-randomizer = "$randlm-dir/buildlm -falsepos 8 -values 8"

```

```

### each language model to be used has its own section here

[LM:thmy]

### command to run to get raw corpus files
#
#get-corpus-script = ""

### raw corpus (untokenized)
#
raw-corpus = $thmy-data/train.$output-extension

### tokenized corpus files (may contain long sentences)
#
#tokenized-corpus = $my-data/train.$output-extension

### if corpus preparation should be skipped,
# point to the prepared language model
#
#lm =

#####
# INTERPOLATING LANGUAGE MODELS

[INTERPOLATED-LM]

# if multiple language models are used, these may be combined
# by optimizing perplexity on a tuning set
# see, for instance [Koehn and Schwenk, IJCNLP 2008]

### script to interpolate language models
# if commented out, no interpolation is performed
#
# script = $moses-script-dir/ems/support/interpolate-lm.perl

### tuning set
# you may use the same set that is used for mert tuning (reference set)
#
#tuning-smg =
#raw-tuning =
#tokenized-tuning =
#factored-tuning =
#lowercased-tuning =
#split-tuning =

### group language models for hierarchical interpolation
# (flat interpolation is limited to 10 language models)
#group = "first,second fourth,fifth"

```

```

### script to use for binary table format for irstlm or kenlm
# (default: no binarization)

# irstlm
#lm-binarizer = $irstlm-dir/compile-lm

# kenlm, also set type to 8
lm-binarizer = $moses-bin-dir/build_binary
type = 8

### script to create quantized language model format (irstlm)
# (default: no quantization)
#
#lm-quantizer = $irstlm-dir/quantize-lm

### script to use for converting into randomized table format
# (default: no randomization)
#
#lm-randomizer = "$randlm-dir/buildlm -falsepos 8 -values 8"

#####
# MODIFIED MOORE LEWIS FILTERING

[MML] IGNORE

### specifications for language models to be trained
#
#lm-training = $srilm-dir/ngram-count
#lm-settings = "-interpolate -kndiscount -unk"
#lm-binarizer = $moses-src-dir/bin/build_binary
#lm-query = $moses-src-dir/bin/query
#order = 5

### in-/out-of-domain source/target corpora to train the 4 language model
#
# in-domain: point either to a parallel corpus
#outdomain-stem = [CORPUS:toy:clean-split-stem]

# ... or to two separate monolingual corpora
#indomain-target = [LM:toy:lowercased-corpus]
#raw-indomain-source = $toy-data/nc-5k.$input-extension

# point to out-of-domain parallel corpus
#outdomain-stem = [CORPUS:giga:clean-split-stem]

# settings: number of lines sampled from the corpora to train each language
model on

```

```

# (if used at all, should be small as a percentage of corpus)
#settings = "--line-count 100000"

#####
# TRANSLATION MODEL TRAINING

[TRAINING]

### training script to be used: either a legacy script or
# current moses training script (default)
#
script = $moses-script-dir/training/train-model.perl

### general options
# these are options that are passed on to train-model.perl, for instance
# * "-mgiza -mgiza-cpus 8" to use mgiza instead of giza
# * "-sort-buffer-size 8G -sort-compress gzip" to reduce on-disk sorting
# * "-sort-parallel 8 -cores 8" to speed up phrase table building
#
#training-options = "-sort-parallel 8 -cores 8"

#training-options = "-cores 8"

### factored training: specify here which factors used
# if none specified, single factor training is assumed
# (one translation step, surface to surface)
#
#input-factors = word lemma pos morph
#output-factors = word lemma pos
#alignment-factors = "word -> word"
#translation-factors = "word -> word"
#reordering-factors = "word -> word"
#generation-factors = "word -> pos"
#decoding-steps = "t0, g0"

### parallelization of data preparation step
# the two directions of the data preparation can be run in parallel
# comment out if not needed
#
parallel = yes

### pre-computation for giza++
# giza++ has a more efficient data structure that needs to be
# initialized with snt2cooc. if run in parallel, this may reduces
# memory requirements. set here the number of parts
#
#run-giza-in-parts = 5

```

```

### symmetrization method to obtain word alignments from giza output
# (commonly used: grow-diag-final-and)
#
alignment-symmetrization-method = grow-diag-final-and

### use of Chris Dyer's fast align for word alignment
#
#fast-align-settings = "-d -o -v"

### use of berkeley aligner for word alignment
#
#use-berkeley = true
#alignment-symmetrization-method = berkeley
#berkeley-train = $moses-script-dir/ems/support/berkeley-train.sh
#berkeley-process = $moses-script-dir/ems/support/berkeley-process.sh
#berkeley-jar = /your/path/to/berkeleyaligner-1.1/berkeleyaligner.jar
#berkeley-java-options = "-server -mx30000m -ea"
#berkeley-training-options = "-Main.iters 5 5 -EMWordAligner.numThreads 8"
#berkeley-process-options = "-EMWordAligner.numThreads 8"
#berkeley-posterior = 0.5

### use of baseline alignment model (incremental training)
#
#baseline = 68
#baseline-alignment-model = "$working-dir/training/prepared.$baseline/$input-
extension.vcb \
# $working-dir/training/prepared.$baseline/$output-extension.vcb \
# $working-dir/training/giza.$baseline/${output-extension}-$input-
extension.cooc \
# $working-dir/training/giza-inverse.$baseline/${input-extension}-$output-
extension.cooc \
# $working-dir/training/giza.$baseline/${output-extension}-$input-
extension.thmm.5 \
# $working-dir/training/giza.$baseline/${output-extension}-$input-
extension.hhmm.5 \
# $working-dir/training/giza-inverse.$baseline/${input-extension}-$output-
extension.thmm.5 \
# $working-dir/training/giza-inverse.$baseline/${input-extension}-$output-
extension.hhmm.5"

### if word alignment should be skipped,
# point to word alignment files
#
#word-alignment = $working-dir/model/aligned.1

### filtering some corpora with modified Moore-Lewis
# specify corpora to be filtered and ratio to be kept, either before or after
word alignment

```

```

#mml-filter-corpora = toy
#mml-before-wa = "-proportion 0.9"
#mml-after-wa = "-proportion 0.9"

### create a bilingual concordancer for the model
#
#biconcor = $moses-script-dir/ems/biconcor/biconcor

#####

### use of Operation Sequence Model
### Durrani, Schmid and Fraser. (2011): "A Joint Sequence Translation Model with
Integrated Reordering"

#operation-sequence-model = "yes"
#operation-sequence-model-order = 5
#operation-sequence-model-settings = ""

### compile Moses with --max-kenlm-order=9 if higher order is required

### if OSM training should be skipped,
# point to OSM Model
#
# osm-model =

#####

### lexicalized reordering: specify orientation type
# (default: only distance-based reordering model)
#
lexicalized-reordering = msd-bidirectional-fe

### hierarchical rule set
#
#hierarchical-rule-set = true

### settings for rule extraction
#
#extract-settings = ""
max-phrase-length = 5

### add extracted phrases from baseline model
#
#baseline-extract = $working-dir/model/extract.$baseline
#
# requires aligned parallel corpus for re-estimating lexical translation
probabilities

```

```

#baseline-corpus = $working-dir/training/corpus.$baseline
#baseline-alignment = $working-dir/model/aligned.$baseline.$alignment-
symmetrization-method

### unknown word labels (target syntax only)
# enables use of unknown word labels during decoding
# label file is generated during rule extraction
#
#use-unknown-word-labels = true

### if phrase extraction should be skipped,
# point to stem for extract files
#
# extracted-phrases =

### settings for rule scoring
#
score-settings = "--GoodTuring"

### include word alignment in phrase table
#
#include-word-alignment-in-rules = yes

### sparse lexical features
#
#sparse-features = "target-word-insertion top 50, source-word-deletion top 50,
word-translation top 50 50, phrase-length"

### domain adaptation settings
# options: sparse, any of: indicator, subset, ratio
#domain-features = "subset"

### if phrase table training should be skipped,
# point to phrase translation table
#
# phrase-translation-table =

### if reordering table training should be skipped,
# point to reordering table
#
# reordering-table =

### filtering the phrase table based on significance tests
# Johnson, Martin, Foster and Kuhn. (2007): "Improving Translation Quality by
Discarding Most of the Phrasetable"
# options: -n number of translations; -l 'a+e', 'a-e', or a positive real value
-log prob threshold
#salm-index = /path/to/project/salm/Bin/Linux/Index/IndexSA.064

```

```

#sigtest-filter = "-l a+e -n 50"

### if training should be skipped,
# point to a configuration file that contains
# pointers to all relevant model files
#
#config-with-reused-weights =

#####
### TUNING: finding good weights for model components

[TUNING]

### instead of tuning with this setting, old weights may be recycled
# specify here an old configuration file with matching weights
#
#weight-config = /panfs/panltg2/users/finch/expt/moses2.1/plain.weight.ini

### tuning script to be used
#
tuning-script = $moses-script-dir/training/mert-moses.pl
tuning-settings = "-mertdir $moses-bin-dir"

### specify the corpus used for tuning
# it should contain 1000s of sentences
#
#input-smg =
raw-input = $thmy-data/dev.$input-extension

#tokenized-input =
#factorized-input =
#input =
#
#reference-smg =
raw-reference = $thmy-data/dev.$output-extension

#tokenized-reference =
#factorized-reference =
#reference =

### size of n-best list used (typically 100)
#
nbest = 100

### ranges for weights for random initialization
# if not specified, the tuning script will use generic ranges
# it is not clear, if this matters
#

```

```

# lambda =

### additional flags for the filter script
#
filter-settings = ""

### additional flags for the decoder
#
decoder-settings = ""

### if tuning should be skipped, specify this here
# and also point to a configuration file that contains
# pointers to all relevant model files
#
#config =

#####
## RECASER: restore case, this part only trains the model

[RECASING]

#decoder = $moses-bin-dir/moses

### training data
# raw input needs to be still tokenized,
# also also tokenized input may be specified
#
#tokenized = [LM:europarl:tokenized-corpus]

# recase-config =

#lm-training = $srilm-dir/ngram-count

#####
## TRUECASER: train model to truecase corpora and input

[TRUECASER]

### script to train truecaser models
#
#trainer = $moses-script-dir/recaser/train-truecaser.perl

### training data
# data on which truecaser is trained
# if no training data is specified, parallel corpus is used
#
# raw-stem =
# tokenized-stem =

```

```

### trained model
#
# truecase-model =

#####
## EVALUATION: translating a test set using the tuned system and score it

[EVALUATION]

### additional flags for the filter script
#
#filter-settings = ""

### additional decoder settings
# switches for the Moses decoder
# common choices:
#   "-threads N" for multi-threading
#   "-mbr" for MBR decoding
#   "-drop-unknown" for dropping unknown source words
#   "-search-algorithm 1 -cube-pruning-pop-limit 5000 -s 5000" for cube pruning
#
decoder-settings = "-search-algorithm 1 -cube-pruning-pop-limit 5000 -s 5000"

### specify size of n-best list, if produced
#
#nbest = 100

### multiple reference translations
#
#multiref = yes

### prepare system output for scoring
# this may include detokenization and wrapping output in sgm
# (needed for nist-bleu, ter, meteor)
#
#detokenizer = "$moses-script-dir/tokenizer/detokenizer.perl -l $output-
extension"
#recaser = $moses-script-dir/recaser/recase.perl
wrapping-script = "$moses-script-dir/ems/support/wrap-xml.perl $output-
extension"
#output-sgm =

### BLEU
#
nist-bleu = $moses-script-dir/generic/mteval-v13a.pl
nist-bleu-c = "$moses-script-dir/generic/mteval-v13a.pl -c"
multi-bleu = $moses-script-dir/generic/multi-bleu.perl

```

```

#ibm-bleu =

### TER: translation error rate (BBN metric) based on edit distance
# not yet integrated
#
# ter =

### METEOR: gives credit to stem / worknet synonym matches
# not yet integrated
#
# meteor =

### Analysis: carry out various forms of analysis on the output
#
analysis = $moses-script-dir/ems/support/analysis.perl
#
# also report on input coverage
analyze-coverage = yes
#
# also report on phrase mappings used
report-segmentation = yes
#
# report precision of translations for each input word, broken down by
# count of input word in corpus and model
report-precision-by-coverage = yes
#
# further precision breakdown by factor
#precision-by-coverage-factor = pos
#
# visualization of the search graph in tree-based models
#analyze-search-graph = yes

[EVALUATION:test]

### input data
#
#input-smg = /panfs/panmt/users/ye/cluster/test-smg/test.$input-
extension.src.sgm
input-smg = $thmy-data/test-smg/test.$input-extension.src.sgm
#input-smg = $mykc-data/test-smg/test.$input-extension.src.sgm

# raw-input = $mykc-data/test.$input-extension
# tokenized-input =
# factorized-input =
# input =

### reference data
#

```

```

#reference-sgm = /panfs/panmt/users/ye/cluster/test-sgm/test.$output-
extension.ref.sgm
reference-sgm = $thmy-data/test-sgm/test.$output-extension.ref.sgm
#reference-sgm = $mykc-data/test-sgm/test.$output-extension.ref.sgm

# raw-reference = $mykc-data/test.$output-extension
# tokenized-reference =
# reference =

### analysis settings
# may contain any of the general evaluation analysis settings
# specific setting: base coverage statistics on earlier run
#
#precision-by-coverage-base = $working-dir/evaluation/test.analysis.5

### wrapping frame
# for nist-bleu and other scoring scripts, the output needs to be wrapped
# in sgm markup (typically like the input sgm)
#
wrapping-frame = $input-sgm

#####
### REPORTING: summarize evaluation scores

[REPORTING]

### currently no parameters for reporting section

Update လုပ်ထားတဲ့ perl မိုင်နှစ်မိုင်ကိုလည်း လေ့လာကြည့်ရအောင်။

```

[40]: `!cat ./generate_configs.pl`

```

#!/usr/bin/perl

# Preparing configuration files for PBSMT experiments
# Output: one configuration file for baseline or PBSMT
# *** Before you run this script, you should prepare training, development and
test parallel-data in advance!!!

# Written by Finch-san and Ye
# We used this script for running our NAACL paper:
# Ye Kyaw Thu, Andrew Finch and Eiichiro Sumita,
# "Interlocking Phrases in Phrase-based Statistical Machine Translation",
# In Proceedings of the NAACL-HLT 2016, June 12-17, San Diego, US, pp.
1076-1081.

# Last updated 9 Oct 2019, @Machine Translation Research Summer School, YTU,
Myanmar

```

```

use strict;

my @langs;

# you have to update the PBSMT experiment path!!!
#my $smtpath = "/home/lar/experiment/kachin-myanmar/demo-mykc-smt";
#my $smtpath = "/media/lar/Transcend/student/lecture/mtrss/pbsmt-demo/pbsmt";
#my $smtpath = "/home/ye/exp/SMT-NMT_tutorial/pbsmt-big";
my $smtpath = "/home/ye/exp/SMT-NMT_tutorial/pbsmt-big-normalize";

# you have to update the data path for running PBSMT experiment!!!

#foreach my $trainFile (
</panfs/panltg2/users/finch/expt/multibtec/corpus/train.[a-z][a-z]> )
#foreach my $trainFile (
</panfs/panmt/users/ye/cluster/corpus/train.[a-z][a-z]>)
#foreach my $trainFile ( </home/tha/experiment/my-rk/data/train.[a-z][a-z]>)
#foreach my $trainFile ( </home/lar/experiment/kachin-myanmar/demo-mykc-
smt/4demo/train.[a-z][a-z]>)
#foreach my $trainFile ( </media/lar/Transcend/student/lecture/mtrss/pbsmt-
demo/pbsmt/data/train.[a-z][a-z]>)
#foreach my $trainFile ( </home/ye/exp/SMT-NMT_tutorial/clean-
data/big/train.[a-z][a-z]>)
foreach my $trainFile ( </home/ye/exp/SMT-NMT_tutorial/clean-
data/big/normalize/data/train.[a-z][a-z]>)
{
    $trainFile =~ m/train.([a-z][a-z])/;

    push @langs, $1;
}

foreach my $expt (qw/baseline/)
{

die("$smtpath/$expt") if (-d "$smtpath/$expt");
`mkdir $smtpath/$expt`;
    foreach my $src (@langs)
    {
        foreach my $trg (@langs)
        {
            next if $src eq $trg;
            #next if $src ne "my" and $trg ne "my";

            my $pair = "${src}-${trg}";
            my $configFile = "$smtpath/$expt/$pair/config.${expt}.$pair";

            `mkdir $smtpath/$expt/$pair`;

```

```

        open FILE, ">$configFile" or die;

        print FILE "# Config file for $pair ($expt)\n";
        print FILE "\n[GENERAL]\n";
        print FILE "working-dir = $smtpath/$expt/$pair\n";
        print FILE "input-extension = ${src}\n";
        print FILE "output-extension = ${trg}\n";
        print FILE "pair-extension = ${pair}\n";
        print FILE "\n";
        close FILE;

        `cat $configFile config.$expt > tmp`;
        `mv tmp $configFile`;
    }
}

```

[41]: !cat run-baseline.pl

```

#!/usr/bin/perl
use strict;
use warnings;

require IPC::System::Simple;
use autodie qw(:all);

# Written by Finch-san and Ye
# We used this script for running our NAACL paper:
# Ye Kyaw Thu, Andrew Finch and Eiichiro Sumita,
# "Interlocking Phrases in Phrase-based Statistical Machine Translation",
# In Proceedings of the NAACL-HLT 2016, June 12-17, San Diego, US, pp.
# 1076-1081.

# Last updated 9 Oct 2019, @Machine Translation Research Summer School, YTU,
# Myanmar

# You have to update following path!!!

#my @configs = `find /home/ros/experiment/langacq/exp1/smt/{baseline,osm,hiero}
-name "config.*" | sort`;
#my @configs = `find /home/ros/experiment/langacq/exp2a/smt/*/* -name
"config.*.*" | sort`;
#my @configs = `find /home/ros/experiment/sl-my/smt1/*/* -name
"config.baseline.*" | sort`;
#my @configs = `find /home/ros/experiment/my-rk/smt1/*/* -name
"config.baseline.*" | sort`;
#my @configs = `find /media/lar/Transcend/exp/kachin-myanmar/demo-mykc-smt/

```

```

-name "config.baseline" `;
#my @configs = `find /home/lar/experiment/kachin-myanmar/demo-mykc-smt/*/ -name
"config.baseline*" | sort`;
#my @configs = `find /media/lar/Transcend/student/lecture/mtrss/pbsmt-
demo/pbsmt/*/ -name "config.baseline*" | sort`;

#my @configs = `find /home/ye/exp/SMT-NMT_tutorial/pbsmt-big/*/ -name
"config.baseline*" | sort`;
my @configs = `find /home/ye/exp/SMT-NMT_tutorial/pbsmt-big-normalize/*/ -name
"config.baseline*" | sort`;

#for debugging
#print "config files: @configs\n";
#exit;

#my @configs = qw(/panfs/panltg2/users/finch/expt/moses2.1/baseline/zh-
nl/config.baseline.zh-nl);
#my @langs = qw(mini_en mini_ja/;

foreach my $i (0..$#configs)
{
    # sleep until there are less than 40 of my jobs on the queue
    #while(`qstat | grep ye | wc` =~ /\s*([0-9]+)/ && $1 > 700 )
    #{
    #    sleep 20;
    #}

#    my $o_config = $o_configs[$i];
#    my $b_config = $b_configs[$i];
#    my $config = $configs[$i];

#    chomp $o_config;
#    chomp $b_config;
#    chomp $config;

    my @toks = split /\./, $config;
    my $jobName = "$toks[$#toks]-$toks[$#toks - 1]";

    print "$jobName $config\n";

    #    `/home/lar/tool/moses/scripts/ems/experiment.perl -config
$config -exec -no-graph 2>&1 | tee -a run1.log`;

#/home/ye/tool/mosesbin/ubuntu-17.04/moses/scripts/ems
    `/home/ye/tool/mosesbin/ubuntu-17.04/moses/scripts/ems/experiment.perl
-config $config -exec -no-graph 2>&1 | tee -a run1.log`;
    #`/home/ros/mosesdecoder/scripts/ems/experiment.perl -config $config
-exec -no-graph`;

```

```

#       my @toks = split /\./, $o_config;
#       my $jobName = "$toks[$#toks]-$toks[$#toks - 1]";
#
#       print "$jobName $o_config\n";
#
#       `qsub -q mt128 -N "$jobName" --
/panfs/panltg2/users/finch/local/mosesdecoder.v21/scripts/ems/experiment.perl
-config $o_config -exec`;
#
#       @toks = split /\./, $b_config;
#       $jobName = "$toks[$#toks]-$toks[$#toks - 1]";
#
#       print "$jobName $b_config\n\n";
#
#       `qsub -q mt128 -N "$jobName" --
/panfs/panltg2/users/finch/local/mosesdecoder.v21/scripts/ems/experiment.perl
-config $b_config -exec`;
}

```

1.8 Run PBSMT Experiment with Normalized Myanmar Data

ဒီကနေစပြီး configuration ဖိုင်ထုတ်မယ်။ ပြီးတာနဲ့ PBSMT experiment ကို မြန်မာစာ normalized လုပ်ထားတဲ့ ဒေတာနဲ့ မြန်မာ-ထိုင်း၊ ထိုင်း-မြန်မာ နှစ်မျိုးစလုံးကို ဘာသာပြန်ကြည့်မယ်။ ရလဒ်က တက်လာမယ်လို့ မျှော်လင့်ပါတယ်။

အရင်ဆုံး configuration ဖိုင်တွေထုတ်ပေးမယ့် generate_configs.pl ကို run ပါမယ်။

[43]: `!perl generate_configs.pl`

အောက်ပါအတိုင်း baseline folder နဲ့ အဲဒီ အောက်မှာ လိုအပ်တဲ့ sub-folder တွေဆောက်ပြီး configuration ဖိုင်တွေကို ပြင်ဆင်ပေးသွားတာကို တွေ့ရပါလိမ့်မယ်။

[44]: `!tree .`

```

.
├── baseline
│   ├── my-th
│   │   └── config.baseline.my-th
│   └── th-my
│       └── config.baseline.th-my
├── config.baseline
├── generate_configs.pl
└── run-baseline.pl

```

4 directories, 5 files

PBSMT experiment ကို စလုပ်မယ်။ ဆရာစက်ထဲမှာတော့ ငှာရီခန့် ကြာမယ်လို့ ခန့်မှန်း

```
[63]: %cd /home/ye/exp/SMT-NMT_tutorial/pbsmt-big-normalize
```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt-big-normalize
```

```
[64]: !time perl ./run-baseline.pl
```

```
my-th-baseline /home/ye/exp/SMT-NMT_tutorial/pbsmt-big-normalize/baseline/my-  
th/config.baseline.my-th  
th-my-baseline /home/ye/exp/SMT-NMT_tutorial/pbsmt-big-normalize/baseline/th-  
my/config.baseline.th-my
```

```
real    556m51.930s  
user    0m24.638s  
sys     0m32.643s
```

အထက်မှာ မြင်ရတဲ့အတိုင်း my-th, th-my မော်ဒယ်နှစ်ခုအတွက်က ငြနာရီခွဲနီးပါး ကြာခဲ့ပါတယ်။
ရလဒ်တွေကို ကြည့်ကြရအောင်။

1.9 BLEU Scores

မြန်မာ-ထိုင်းအတွက်က

```
[66]: %cd /home/ye/exp/SMT-NMT_tutorial/pbsmt-big-normalize/baseline/my-th
```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt-big-normalize/baseline/my-th
```

```
[68]: !ls --color=auto
```

```
config.baseline.my-th  corpus  evaluation  
lm  model  steps  training  
tuning
```

```
[69]: %cd evaluation
```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt-big-normalize/baseline/my-th/evaluation
```

```
[71]: !ls --color=auto
```

```
test.analysis.1      test.filtered.ini.1  test.nist-bleu-c.1  
test.cleaned.1       test.input.txt.1     test.output.1  
test.detokenized.sgm.1 test.multi-bleu.1    test.output.1.wa  
test.filtered.1      test.nist-bleu.1     test.reference.txt.1
```

```
[72]: %cat test.multi-bleu.1
```

```
BLEU = 37.43, 58.1/40.4/31.7/26.4 (BP=1.000, ratio=1.003, hyp_len=56305,  
ref_len=56113)
```

ထိုင်း-မြန်မာ ဘာသာပြန်အတွက် test result က အောက်ပါအတိုင်းပါ။

```
[73]: %cd /home/ye/exp/SMT-NMT_tutorial/pbsmt-big-normalize/baseline/th-my/evaluation
```

```
/home/ye/exp/SMT-NMT_tutorial/pbsmt-big-normalize/baseline/th-my/evaluation
```

```
[75]: !ls --color=auto
```

```
test.analysis.1      test.filtered.ini.1  test.nist-bleu-c.1
test.cleaned.1       test.input.txt.1    test.output.1
test.detokenized.sgm.1 test.multi-bleu.1    test.output.1.wa
test.filtered.1      test.nist-bleu.1     test.reference.txt.1
```

```
[76]: !cat ./test.multi-bleu.1
```

```
BLEU = 32.53, 60.1/39.3/26.9/19.3 (BP=0.978, ratio=0.978, hyp_len=90210,
ref_len=92231)
```

1.10 Summary

Normalization မလုပ်ခင်က ရခဲ့တဲ့ ရလဒ်တွေကို baseline ထားပြီး လက်ရှိ normalized လုပ်ထားတဲ့ မြန်မာစာနဲ့ run ပြီးရလာတဲ့ ရလဒ်တွေကို နှိုင်းယှဉ်ကြည့်ရင် အောက်ပါဇယားအတိုင်း ရပါတယ်။

Experiment	Direction	BLEU	1-gram	2-gram	3-gram	4-gram	BP	Ratio
Exp-1: Baseline	my →	36.80	57.3	39.7	31.1	25.9	1.000	1.004
	th							
	th →	32.76	59.7	39.4	27.4	19.9	0.973	0.974
Exp-2: Normal- ized	my							
	my →	37.43	58.1	40.4	31.7	26.4	1.000	1.003
	th							
	th →	32.53	60.1	39.3	26.9	19.3	0.978	0.978
	my							

ဒီရလဒ်ကနေ Myanmar → Thai ဘာသာပြန်တာမှာတော့ (+0.63 BLEU) ဖြစ်လာတာကို ရှင်းရှင်းလင်းလင်း မြင်တွေ့ရပါတယ်။ သို့သော်လည်း စိတ်ဝင်စားဖို့ကောင်းတာက ထိုင်းကနေ မြန်မာဘက်အခြမ်းမှာတော့ (-0.23 BLEU) ဖြစ်သွားတာကို တွေ့ရှိရပါတယ်။ ဒီလိုပါပဲ ဒေတာတိုးတာမျိုး မဟုတ်ပဲနဲ့ BLEU score ကို 1.0 လောက်တက်လာဖို့က လက်တွေ့မှာ ခက်ပါတယ်။ တခြား combination တွေ လုပ်ကြည့်ရင် ရလဒ်က လက်ရှိထက်တော့ တက်လာနိုင်ပါသေးတယ်။

နိဂုံးချုပ်အနေနဲ့ Machine Translation ကို ထဲထဲဝင်ဝင် ကျွမ်းကျင်ဖို့က အချိန်အများကြီးပေးပြီး လေ့လာရပါတယ်။ တစ်ခု ကောင်းတာကတော့ SMT က မော်ဒယ် သုံးမျိုး၊ လေးမျိုးတွဲပြီး run ပုံစံမျိုး ဖြစ်တာကြောင့် အဲဒီ မော်ဒယ် တစ်ခုချင်းစီကို (i.e. alignment model, translation model, language model, optimization model, reordering model) အသေအချာ နားလည်သွားရင် ML/AI Engineering အလုပ်တွေ အတွက် အများကြီး အထောက်အကူဖြစ်ပါလိမ့်မယ်။

ဒီ SMT Notebook တွေကို ဆရာ အချိန်ပေးပြီး လက်တွေ့လုပ်ပြထားတာမို့ တကယ်ကြိုးစားလေ့လာတဲ့သူတွေအတွက်က အကျိုးရှိပါလိမ့်မယ်။

```
[ ]:
```